RV UNIVERSITY, BENGALURU

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A Project Report On

EASYPARK - SMART PARKING SYSTEM

B.Tech(Honors)

In

School of Computer Science and Engineering

Submitted By

Team Member 01: 1RUA24CSE0265 - MOULYA N
Team Member 02: 1RUA24CSE0272 - NAMA SHRITHA
Team Member 03: 1RUA24CSE0287 - NIREEKSHA P RATHOD
Team Member 04: 1RUA24CSE0302 - POOJA NARESH M

Course Name: Internet of Things [CS2404]

Under the Guidance of

Sheela S
Assistant Professor
School of CSE
RV University,
Bengaluru-560059 2025-2026

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled **EasyPark - Smart Parking System** is carried out by **Moulya.N** (1RUA24CSE0265), **Nama Shritha** (1RUA24CSE0272), **Nireeksha P Rathod** (1RUA24CSE0287) and **Pooja Naresh M** (1RUA24CSE0302) RV University, Bengaluru, **B.Tech (Hons) in the School of Computer Science and Engineering** of the RV University, Bengaluru during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the project report. The Project report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution.

Signature of Guide

External Viva:

Name of Examiners **Signature with Date**

1

2

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, Moulya N(1RUA24CSE0265), Nama Shritha(1RUA24CSE0272), Nireeksha P Rathod (1RUA24CSE0287)and Pooja Naresh M(1RUA24CSE0302) students of second semester B.Tech(Hons), SoCSE, RV University, Bengaluru, hereby declare that the project titled **EasyPark - Smart Parking System** has been carried out by us and submitted in partial fulfillment of **Bachelor of Technology(Hons)** in **School of Computer Science and Engineering** during the year 2025-26.Further, we declare that the content of the report has not been submitted previously by anybody or to any other university. We also declare that any Intellectual Property Rights generated out of this project carried out at RV University will be the property of RV University, Bengaluru, and we will be one of the authors of the same.

Place:
Bengaluru Date:

Name **Signature**

1. Moulya N (1RUA24CSE0265)

2. Nama Shritha (1RUA24CSE0272)

3. Nireeksha P Rathod (1RUA24CSE0287)

4. Pooja Naresh M (1RUA24CSE0302)

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project.First, we take this opportunity to express our sincere gratitude to the School of Computer Science and Engineering, RV University, for providing us with a great opportunity to pursue our bachelor's degree in this institution.A special thanks to our Program Director, **Dr.Ashwini K, and** Dean - **Dr.Shobha G,** for their continuous support and providing the necessary facilities with guidance to carry out mini project work.We would like to thank our guide, Prof**, Sheela S Assistant Professor**, School of Computer Science and Engineering, RV University, for sparing his/her valuable time to extend help in every step of our project work, which paved the way for smooth progress and fruitful culmination of the project. We are also grateful to our family and friends who provided us with every requirement throughout the course.We would like to thank one and all who directly or indirectly helped us in the Project work.

Signature of Students

1RUA24CSE0265:
1RUA24CSE0272:
1RUA24CSE0287:
1RUA24CSE0302:

Abstract

In today's rapidly urbanizing world, **parking space management** has become a major challenge in both metropolitan and developing cities. A considerable amount of time, fuel, and effort is wasted by drivers searching for available parking spaces, leading to increased **traffic congestion**, **air pollution**, and **driver frustration**. To address these inefficiencies, the project ***EasyPark – A Smart Parking System*** introduces an **IoT-based automated solution** that leverages **sensors**, **microcontrollers**, and **real-time web monitoring** to optimize parking management.

The **EasyPark system** integrates **ultrasonic sensors** and **LED indicators** controlled through an **ESP32 microcontroller** to detect the presence or absence of vehicles in each slot. Each slot is visually represented with distinct colors: **Green (Empty)**, **Yellow (Reserved)**, and **Red (Occupied)**. These statuses are continuously updated on an **interactive web dashboard** built using **HTML, CSS, JavaScript, and PHP**, enabling administrators to view live slot conditions remotely. The ESP32 communicates through **Wi-Fi connectivity** with a **PHP proxy server**, ensuring secure and efficient data exchange between the hardware and the web application.

To further enhance the system's functionality, a **reservation feature** is implemented that allows users to pre-book parking slots for specific durations. This prevents unauthorized use of reserved spaces and ensures **fair resource allocation**. The hardware continuously monitors the parking environment, automatically detecting vehicle movements and updating slot statuses in real time. By combining **hardware automation** with **intelligent software interfaces**, EasyPark achieves a seamless integration of physical and digital components, significantly improving **operational efficiency**, **user convenience**, and **system reliability**.

In addition, the EasyPark project emphasizes **scalability and modularity**, allowing new parking slots or sensors to be added easily without redesigning the system. The architecture supports future upgrades such as **mobile app integration**, **automated billing**, and **license plate recognition** using image processing. This flexibility ensures that the system remains relevant and adaptable to evolving smart city infrastructures. The design also supports multiple communication protocols, enabling expansion across large parking facilities and integration with municipal management platforms.

From an environmental and economic perspective, the proposed model offers substantial benefits by reducing unnecessary fuel consumption and minimizing idle time spent searching for parking. It aligns with the goals of **sustainable urban development**, promoting **energy efficiency**, **reduced carbon emissions**, and **optimized land use**. Moreover, businesses and municipalities can leverage EasyPark to generate insights on parking usage patterns, supporting **data-driven decisions** for urban planning and resource management.

Ultimately, **EasyPark** demonstrates how **IoT and embedded systems** can transform conventional parking methods into **smart, automated, and eco-friendly solutions**. By integrating real-time sensing, wireless communication, and web-based control, the system creates a unified digital ecosystem for parking management. This innovation not only enhances user experience but also contributes to the vision of **smarter, safer, and more sustainable cities**, making EasyPark a practical step toward a **connected urban future**.

Table of Contents:

List of Tables and Figures

EASYPARK - SMART PARKING SYSTEM

CHAPTER 1 : INTRODUCTION

1.1 General Introduction

In modern cities, finding a parking space has become one of the most frustrating daily challenges faced by vehicle owners. As the number of cars on the road continues to increase, the demand for parking spaces has grown dramatically, while the available parking areas have remained limited. Drivers often spend several minutes or even hours searching for a free spot, which leads to traffic congestion, unnecessary fuel consumption, and air pollution. The lack of an efficient parking management system is a major cause of this problem.

To overcome these challenges, the concept of a **smart parking system** has emerged. A smart parking system uses **modern technologies such as sensors, microcontrollers, and wireless communication** to monitor parking spaces and provide live updates about their status. It not only detects whether a parking slot is empty or occupied but can also allow users to reserve a slot in advance. This helps save time, reduces traffic jams, and makes parking management more organized.

Our project, ***EasyPark – A Smart Parking System***, is designed to automate and simplify the entire parking process. It continuously monitors the parking slots and displays their real-time status using a web interface. Each slot is equipped with a sensor that detects whether a car is present or not. The information is then processed by a controller, and the status of each slot—**empty, reserved, or occupied**—is updated on a web page using color indicators (green for empty, yellow for reserved, and red for occupied). The system also features an admin dashboard where parking managers can monitor all slots at once and manage reservations.

The purpose of EasyPark is to bring **convenience, transparency, and efficiency** into parking systems. It demonstrates how **technology and automation** can improve daily life by reducing stress, saving time, and contributing to smarter cities. Through this project, we aim to provide an affordable, scalable solution that can be implemented in malls, offices, and public parking areas to create a smoother parking experience for everyone.

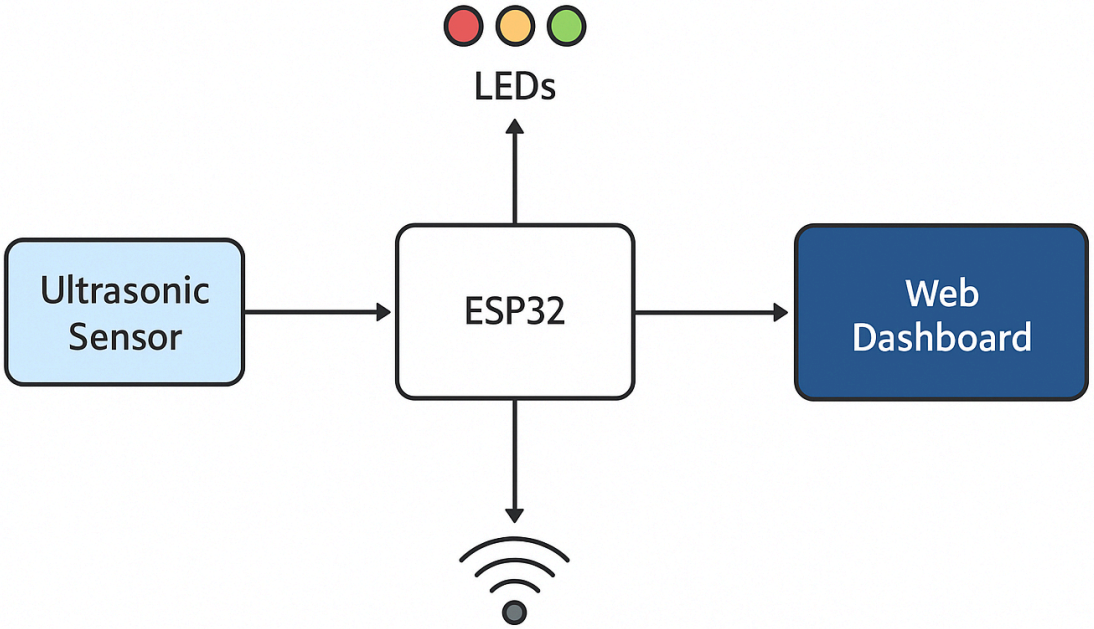


Figure 1.1 System Overview of EasyPark Smart Parking System

1.2 Literature Survey

Over the years, researchers and developers have worked on several approaches to create **automated parking systems**. Traditional parking relied entirely on human supervision—guards would manually check which slots were available and guide drivers to them. However, such methods were prone to errors, time-consuming, and required significant manpower. With the rise of **automation and the Internet of Things (IoT)**, new methods have been developed to make parking systems intelligent and self-managing.

Early smart parking systems used simple sensors and microcontrollers like Arduino to detect the presence of vehicles. For example, some research used **infrared sensors** that detect when a car blocks the sensor's light beam. Others used **ultrasonic sensors**, which measure the distance between the sensor and the vehicle to detect occupancy. These systems could show which slots were available through small display screens placed at the entrance of the parking lot.

However, while these early systems were helpful, they lacked remote accessibility and real-time communication with users. Recent advancements introduced **Wi-Fi enabled microcontrollers** such as the ESP32 and ESP8266, which allow parking data to be transmitted directly to a web server. This means the slot's status can now be viewed online, from anywhere, on a computer or mobile phone. Modern systems also include **LED indicators** to visually show slot status and **reservation features**, allowing users to book slots before arriving.

EasyPark builds upon these developments by combining all these features into one integrated system. It uses the **ESP32** for wireless connectivity, **ultrasonic sensors** for accurate detection, and a **web-based dashboard** for both users and administrators. The system is designed to provide **real-time updates every few seconds**, ensuring that the digital display always matches the actual physical condition of each parking slot. Compared to earlier models, EasyPark emphasizes **user-friendliness, reliability, and scalability**, making it suitable for both small and large parking facilities. It shows how affordable and accessible technology can make city infrastructure smarter and more responsive to everyday needs.

Reference/Project	**Technology/Platform Used**	**Features Provided**	**Limitations Noted**	**How EasyPark Improves**
Manual Parking (Traditional System)	Human Supervision, Signboards	Basic slot allocation, Visual guidance	High error rate, No real-time updates, Inefficient	Automated,Real-time update, IoT based
Early Sensor-Based Systems	Arduino, IR Sensors, Magnetic Sensors	Electronic detection, Some visual displays	No web updates, No reservation, Local-only access	AddsWi-Fi,Live web dashboard, Reservations
Infrared/Ultrasonic Sensor Systems	Arduino/Raspberry Pi + Ultrasonic/IR Sensors	Accurate occupancy detection, Lower maintenance	Offline-only, No remote reservation	Internet-enabled, Remote monitoring
IoT Smart Parking (ESP8266/ES	ESP8266/ESP32, Wi-Fi, Cloud or Web Server, Mobile/Web	Cloud-based access, Real-time	Mostly lacked reservation,admin	AddsAdmin Dashboard,Reser

P32)	App	updates, Multi-slot support	dashboard	vation module
GSM-Based Smart Parking	Arduino/ESP, GSM Module	Remote notification via SMS	Limited to SMS, No Online/Live dashboard	Online UIinstead of SMS only
Recent Research (2022–2025)	Embedded IoT(ESP32/NodeMCU),Web Interface, Mobile Apps	User app for search/book,LED status, Data analytics	Often lacked fullautomationand multi-user admin	Full LED automation, Multi-user/admin support
EasyPark (Your Project)	ESP32,Ultrasonic sensors,LEDs, Web (PHP/MySQL/JS)	Live dashboard,LED-coded slots,Automated reservation,Scalable design	-	All features integrated, reconfigurable, and scalable

Sl. No	**Paper Title**	**Authors**	**Year**	**Platform / Technology Used**	**Key Features**	**Limitation**	**How EasyPark Improves**

1	**Smart Parking System using Arduino and Ultrasonic Sensor**	Bhavya, N.; Kumar, A.; & Raj, P.	2019	Arduino, Ultrasonic Sensor	Simple car detection, LED indicators	No web interface or live updates	Added Wi-Fi (ESP32) & web dashboard for real-time data
2	**IoT Based Smart Parking System Using NodeMCU**	N. Sharma, V. Gupta, & A. Kumar	2020	NodeMCU, IR Sensors, Firebase	Real-time monitoring via cloud	No reservation or admin module	Added booking & admin dashboard
3	**Automated Smart Parking System using Raspberry Pi and Sensors**	R. Patel, S. Sahu, & K. Sharma	2021	Raspberry Pi, Ultrasonic Sensor, Web Server	Slot monitoring & online display	Costly hardware, local use only	Used ESP32 for cost-effective scalability
4	**IoT-Based Smart Parking System Using ESP8266**	S. Gupta & R. Jain	2022	ESP8266, Wi-Fi, Web App	Remote slot monitoring	No reservation system	Introduced reservation module with LED indication
5	**Design and Implementation of IoT Based Smart Parking System Using ESP32**	P. S. Reddy, A. Mehta, & K. Verma	2023	ESP32, Ultrasonic Sensors, Web (PHP/MySQL)	Real-time slot updates, low power	Limited multi-user admin control	Added full admin control, multi-user access, scalability

6	**EasyPark – Smart Parking System** *(Our Project)*	Moulya N., Nama Shritha, Nireeksha P. Rathod, & Pooja Naresh M.	2025	ESP32, Ultrasonic Sensors, LEDs, Web Dashboard	Integrated IoT automation, real-time dashboard, scalable	—	Complete integration of hardware + software for real-time smart parking

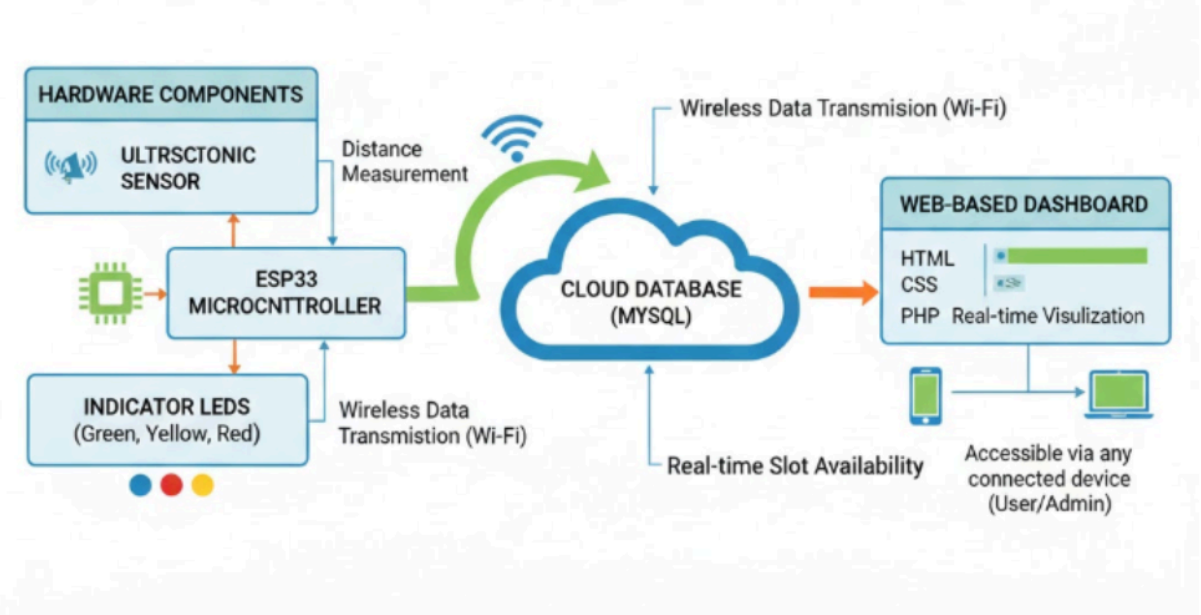


Figure 1.2 EasyPark Smart Parking System Core Components

1.3 Problem Statement & Objectives

In today's rapidly growing urban environments, parking management is a significant issue. Most public and private parking facilities still rely on manual monitoring, where guards check for available slots or drivers search for empty spaces on their own. This outdated approach often leads to **confusion, wasted time, and frustration**. Drivers have no real-time information about slot availability, and administrators lack an efficient way to monitor and control parking activity. This can cause **traffic congestion**, **double parking**, and even **conflicts** between users.

Moreover, there is no standardized system to **reserve parking slots in advance**, which leads to uncertainty and inefficiency. The absence of a live monitoring system also means that management cannot analyze slot usage or detect unauthorized parking. As cities continue to expand, this issue

will only worsen unless a technological solution is introduced.

Objectives

The primary objective of **EasyPark** is to create an intelligent, automated, and user-friendly parking management system that addresses the problems mentioned above. The system's main goals are:

1. To design a **smart parking solution** that automatically detects slot status using sensors.

2. To use a **Wi-Fi-enabled controller (ESP32)** for continuous communication between the parking hardware and the web dashboard.

3. To develop an **interactive admin dashboard** that displays live slot statuses using clear color indicators (green for empty, yellow for reserved, red for occupied).

4. To enable **real-time data updating**, so that any change in slot occupancy is instantly reflected on the website.

5. To incorporate a **reservation feature** allowing users to pre-book slots, which improves time efficiency and organization.

6. To reduce traffic congestion, save fuel, and improve parking management in crowded areas.

7. To create a **scalable model** that can be implemented in various environments such as shopping malls, campuses, offices, or public parking zones.

In short, EasyPark aims to transform the traditional parking experience into a **smart, efficient, and environmentally friendly** system that benefits both drivers and facility managers. It demonstrates how IoT-based automation can solve everyday urban problems by merging simple hardware components with intelligent software design.Over the years, researchers and developers have worked on several approaches to create **automated parking systems**.

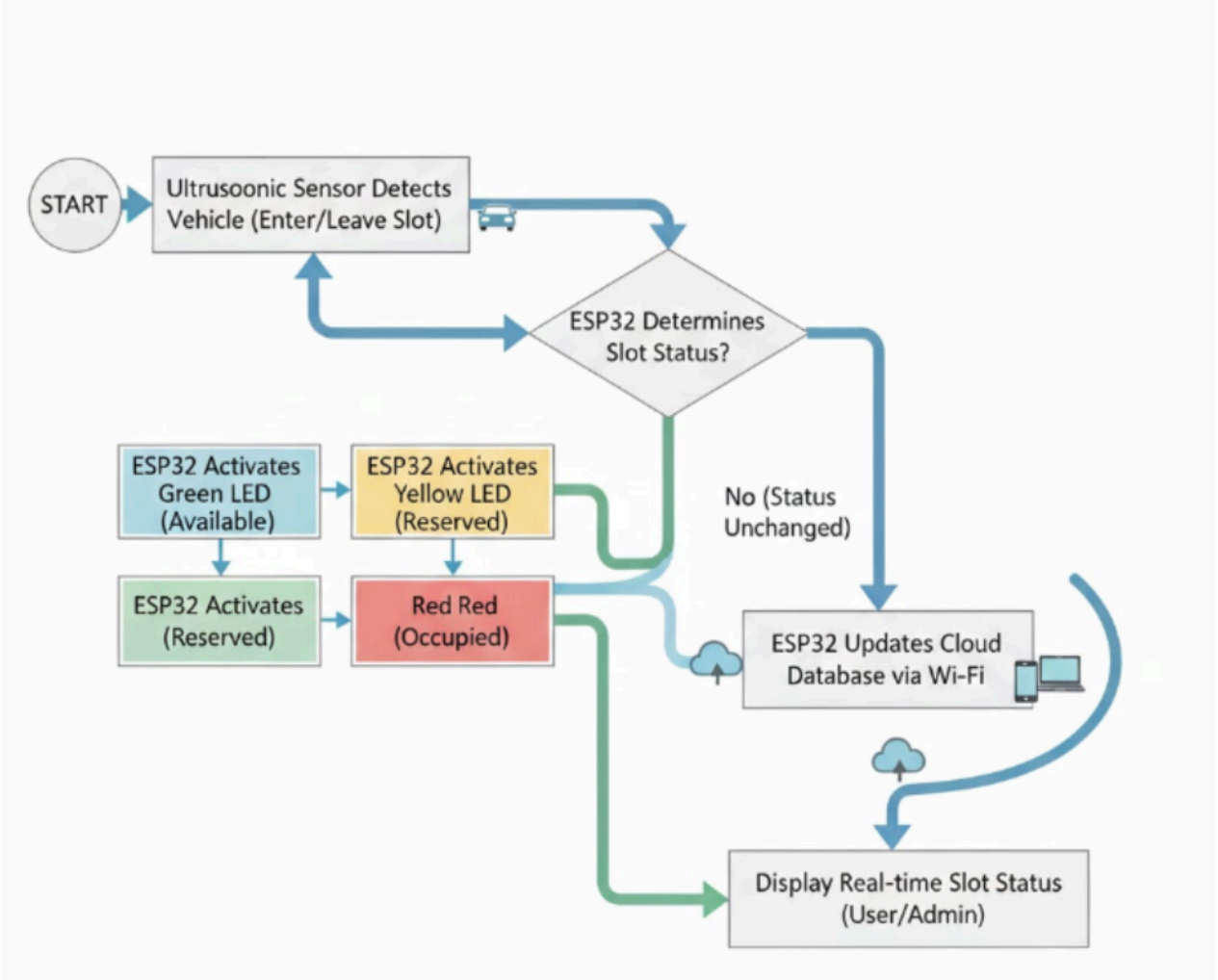


Figure 1.3: Workflow of EasyPark Smart Parking System

CHAPTER 2 : SYSTEM DESIGN

2.1 Architectural Diagram

The **Architectural Diagram** of *EasyPark: A Smart Parking System* represents the **overall system design**, showcasing how the **hardware**, **software**, and **network components** interact to deliver a seamless smart parking experience. The system follows a **three-tier architecture** comprising the **sensing layer**, **control layer**, and **application layer**.

1. **Sensing Layer:**

   This is the **hardware level**, consisting of **ultrasonic sensors** installed at each parking slot. These sensors detect whether a vehicle is present or absent by measuring the distance from the sensor to the object (car). Based on these measurements, the sensor determines the slot's current status as *empty*, *reserved*, or *occupied*.

2. **Control Layer:**

   At the heart of the system lies the **ESP32 microcontroller**, which acts as the **central processing and communication hub**. It collects input data from the sensors, processes the readings, and controls the **LED indicators** to display real-time parking status.

   Additionally, the ESP32 is connected to a Wi-Fi network, allowing it to **transmit live data** to the **web server**. This makes the system intelligent and interconnected, bridging the gap between the physical parking area and the digital platform.

3. **Application Layer:**

   This layer involves the **web-based dashboard** accessible by both administrators and users. The dashboard, designed using **HTML, CSS, JavaScript, and PHP**, retrieves live data from the server (via the ESP32 or a proxy script) and displays it visually.

   The admin can monitor slot statuses, reservations, and overall parking performance, while users can check real-time availability and reserve slots before arriving. The data is stored and managed in a **MySQL database**, ensuring reliability and persistence.

This **architecture** demonstrates the integration of **IoT**, **cloud connectivity**, and **web development** to form a comprehensive parking solution. By uniting these technologies, EasyPark achieves **automation**, **real-time monitoring**, and **remote accessibility**, significantly improving parking efficiency and user convenience.

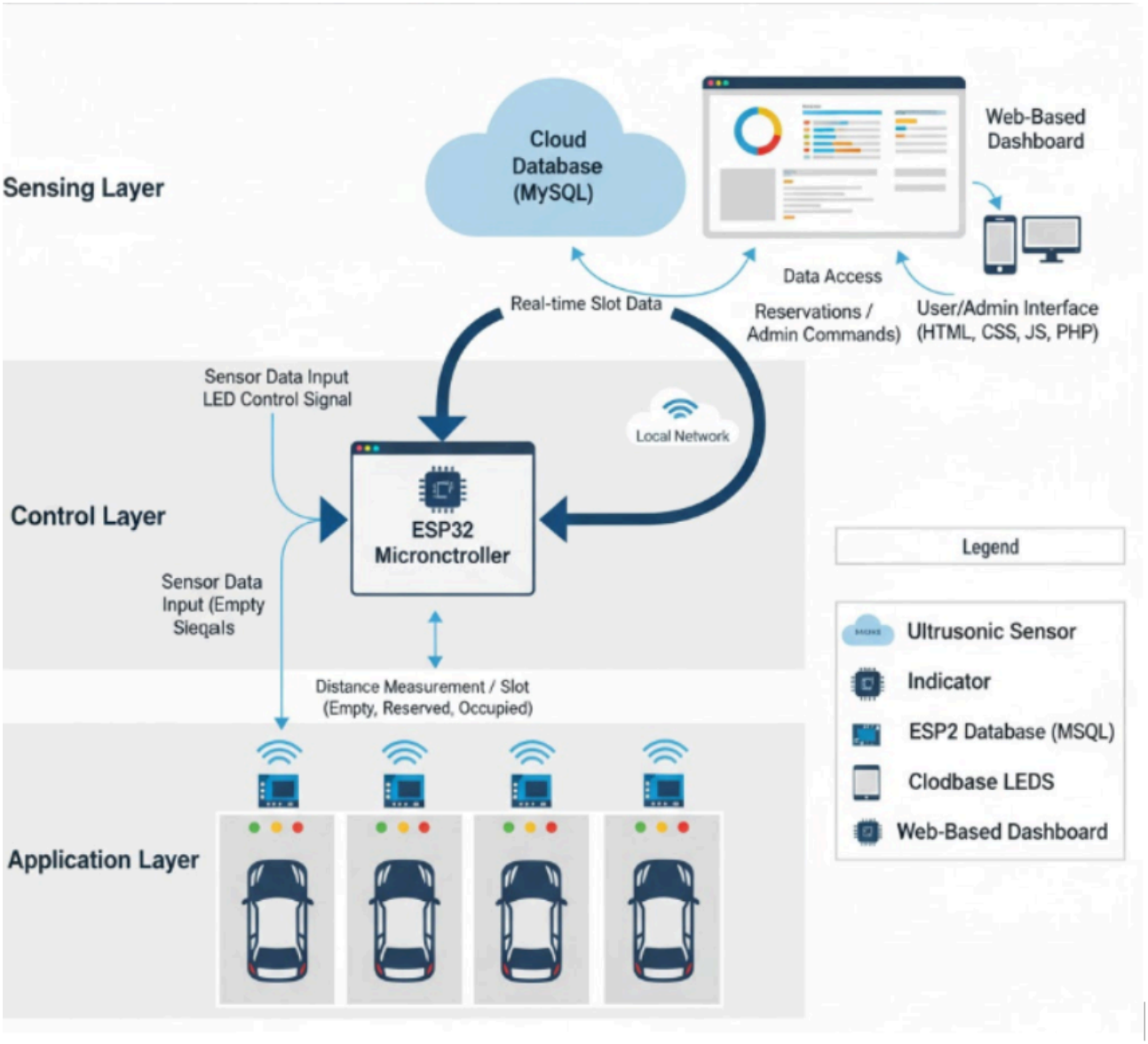


Figure 2.1 Architecture Diagram Of EasyPark: A Smart Parking System

2.2 ER Modelling / ER Schema Diagram

The **Entity–Relationship (ER) Model** is a **conceptual representation** of data that defines how information in a database is logically connected. It provides a **visual overview of entities, their attributes, and the relationships** between them. In the context of *EasyPark: A Smart Parking System*, the ER diagram is crucial in organizing data related to **reservations**, **users**, and **administrators**, ensuring that data is structured, consistent, and easily retrievable.

The **ER Model** for EasyPark consists of **two main entities**:

1. **Venue_admins**
2. **Reservations**

1. Entity: venue_admins

This entity represents the **administrators** responsible for managing parking venues. Each admin can access the dashboard, view slot statuses, and approve or modify reservations.
The attributes of the venue_admins entity include:

- **id** *(INT, Primary Key)* – A unique identifier assigned automatically to each admin.
- **venue_name** *(VARCHAR(100))* – The name of the parking area or venue the admin manages.
- **email** *(VARCHAR(100))* – The registered email address used for authentication.
- **password** *(VARCHAR(255))* – The encrypted password ensures secure login.

This entity ensures **secure access control** and helps maintain accountability for every administrative action performed within the system.

2. Entity: reservations

The **reservations** entity stores all booking and parking details. Each record represents a user's request or confirmed slot reservation in the parking system.
The attributes of this entity are:

- **id** *(INT, Primary Key)* – A unique identifier for every reservation entry.
- **full_name** *(VARCHAR(100))* – The name of the person reserving the slot.
- **contact** *(VARCHAR(15))* – The contact number for communication and confirmation.
- **location** *(VARCHAR(50))* – The parking venue or section where the slot is located.
- **date** *(DATE)* – The booking date for the parking slot.
- **time** *(TIME)* – The starting time of the parking duration.
- **duration** *(INT(11))* – The time (in hours) for which the slot is reserved.

- **assigned_slot** *(VARCHAR(20))* – The specific slot number allocated to the user (e.g., Slot 1, Slot 2).

- **created_at** *(TIMESTAMP)* – Records when the reservation was created for tracking and analytics.

This entity supports **real-time updates** and ensures all reservation data is properly stored and managed through the admin interface.

Relationship Between Entities:

In this version of the EasyPark system, there exists an **implicit relationship** between the **venue_admins** and **reservations** entities.

Each *venue_admin* manages the reservations made within their assigned parking venue. Conceptually, this forms a **one-to-many (1:N)** relationship, where:

- One **venue_admin** can manage **many reservations**,

- But each **reservation** is associated with exactly **one venue_admin**.

Although the current schema doesn't explicitly show a foreign key, it can be extended later by linking **venue_admins.id** to a **foreign key in reservations**, ensuring referential integrity.

ER Model Explanation:

The **ER Schema Diagram (Figure 2.2)** visually represents how these entities interact. Rectangles denote the entities (venue_admins and reservations), ovals represent their attributes, and lines depict relationships. This structure allows developers and database designers to understand **how data flows** between modules and how it is stored for easy retrieval and reporting.

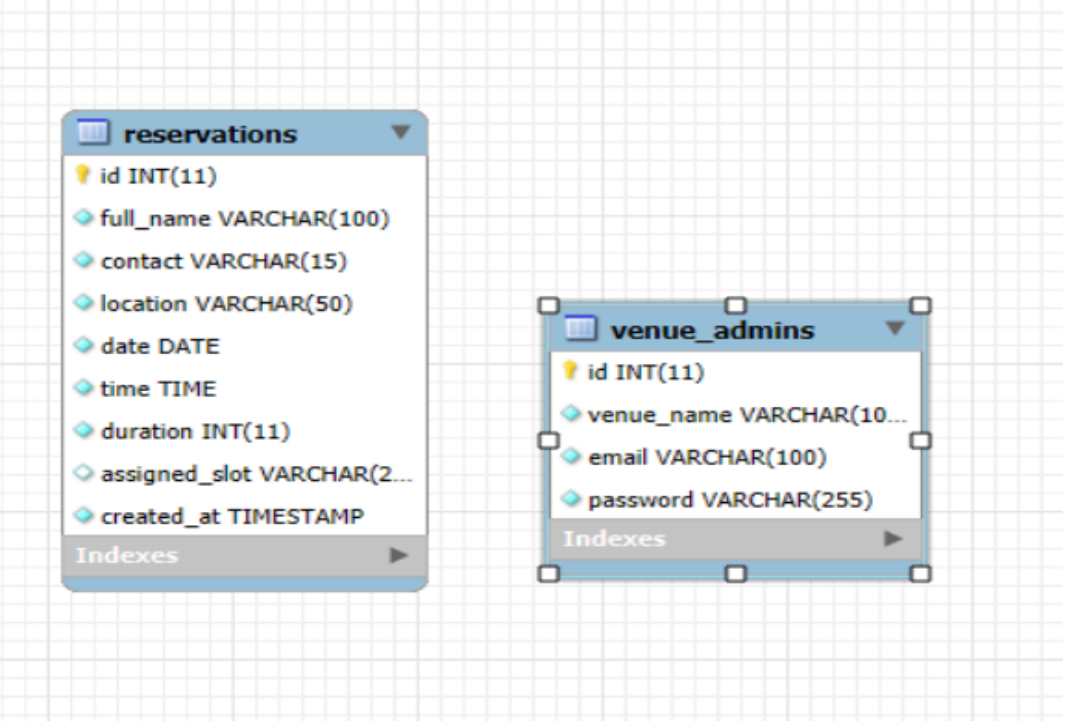


Figure 2.2 – ER Schema Diagram for EasyPark Smart Parking System

CHAPTER 3 : SOFTWARE REQUIREMENTS

3.1 Functional Requirements

Functional requirements define what the system **should do** — the core operations that make EasyPark a fully functional **IoT-based smart parking system**. These requirements ensure that both hardware and software components interact seamlessly to deliver efficient real time parking management.

1. **Admin Login & Authentication** – The system must provide secure access to authorized venue administrators via username and password validation.

2. **Slot Monitoring** – Each parking slot must be continuously monitored using **ultrasonic sensors** connected to the ESP32 microcontroller.

3. **LED Status Indication** – The ESP32 must control LEDs (Green for *Empty*, Yellow for *Reserved*, Red for *Occupied*) to visually represent slot status.

4. **Reservation Module** – Users can reserve parking slots through the frontend website. Once reserved, the system must mark the slot as **"Reserved"** both in the database and on the hardware.

5. **Live Dashboard Display** – The Admin Dashboard should fetch slot data from the ESP32 through a **proxy PHP script** every 2 seconds, ensuring live updates without delay.

6. **Automatic Status Update** – When a vehicle is detected, the sensor should automatically update the slot's status on both hardware (LED change) and software (dashboard update).

7. **Logout Functionality** – The admin must be able to safely log out and be redirected to the login page.

These functional requirements collectively ensure a **real-time, reliable, and user-friendly parking solution** that enhances parking efficiency and reduces human intervention.

3.2 Non-Functional Requirements

Non-functional requirements describe **how well** the system performs its functions, focusing on performance, usability, and maintainability.

1. **Performance:**
   The system should update slot statuses within **2 seconds** to reflect real-world changes instantly. Communication between ESP32 and web server should remain stable even with multiple slots.

2. **Reliability:**
   EasyPark must ensure consistent performance. Even if the ESP32 temporarily loses connection, the system should display “Disconnected” rather than false data.

3. **Scalability:**
   The system should support expansion for additional parking slots or venues with minimal code modification.

4. **Usability:**
   The Admin Dashboard and user interface should be simple, visually appealing, and accessible to non-technical users.

5. **Security:**
   Admin credentials must be stored securely using **hashed passwords** (e.g., password_hash() in PHP). Communication with the server should prevent unauthorized access.

6. **Maintainability:**
   The system’s modular design (ESP32 code, PHP backend, and frontend HTML) ensures easy maintenance and updates.

7. **Availability:**
   The system must be operational **24/7**, with minimal downtime for maintenance.

Overall, the non-functional aspects ensure that EasyPark is not only efficient but also **secure, stable, and sustainable** for long-term use.

3.3 Hardware Requirements

The hardware components used in EasyPark are carefully selected to ensure reliability and cost-effectiveness.

Component	Description
ESP32 Microcontroller	Serves as the central controller; processes sensor input and updates LED indicators. It also communicates with the web server using Wi-Fi.
Ultrasonic Sensors (HC-SR04)	Detects vehicle presence by measuring distance. If the measured distance is below a threshold, the slot is marked as occupied.
LED Indicators	Represent slot status: Green = Empty, Yellow = Reserved, Red = Occupied.
Breadboard and Jumper Wires	Used for circuit connections and testing.
Resistors	Maintain current balance across the LED terminals.
Power Supply (5V)	Powers ESP32 and sensors efficiently.

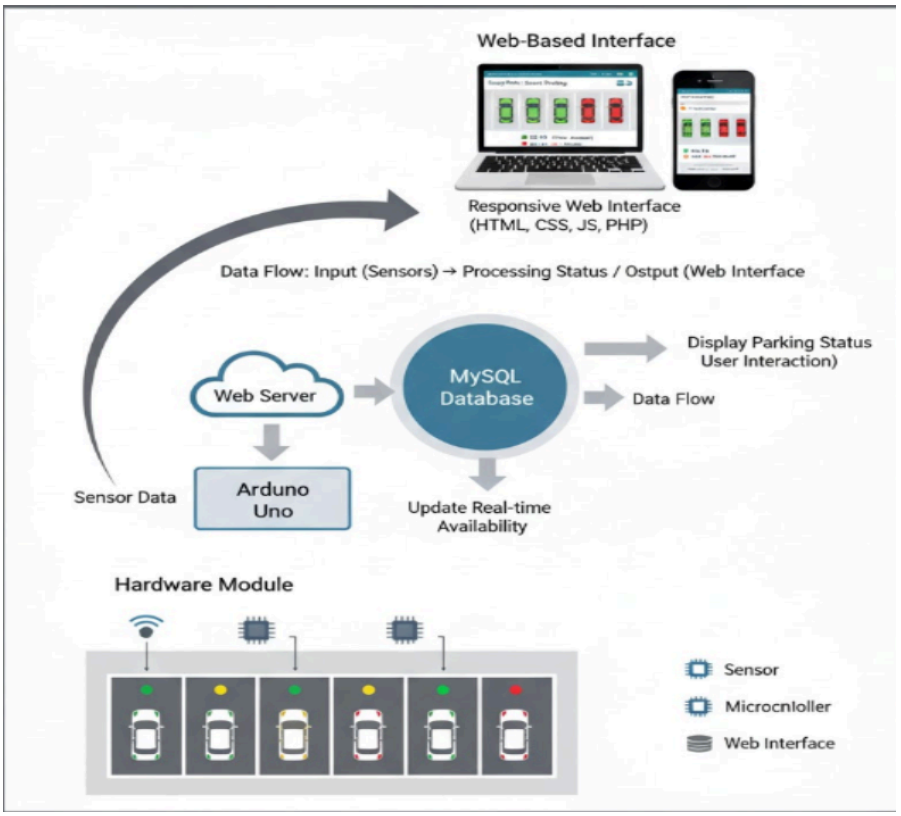


Figure 3.1: Overall Architecture of EasyPark Smart Parking System

3.4 Software Requirements

Software	Purpose
Arduino IDE	To write, compile, and upload ESP32 code.
XAMPP Server	Provides Apache and MySQL to host the PHP backend locally.
PHP & MySQL	Used for backend logic and database management.
HTML, CSS, JavaScript	Used to design the Admin Dashboard and frontend reservation page.
ESP32 Web Server Library	Enables HTTP communication between ESP32 and PHP server.
phpMyAdmin	GUI to manage MySQL database tables.

3.5 Working Principle / Block Diagram

The EasyPark system functions on the principle of **real-time IoT communication** between sensors, microcontroller, and web server.

1. **Vehicle Detection:** The ultrasonic sensor continuously measures distance.

2. **Signal Processing:** The ESP32 analyzes sensor data and determines whether the slot is empty, occupied, or reserved.

3. **LED Indication:** Corresponding LEDs light up based on the slot status.

4. **Data Transmission:** The ESP32 sends the slot status to the server using HTTP requests.

5. **Database Update:** The PHP backend updates the MySQL database with the latest slot statuses.

6. **Admin Dashboard:** The web interface fetches and displays live slot data every 2 seconds.

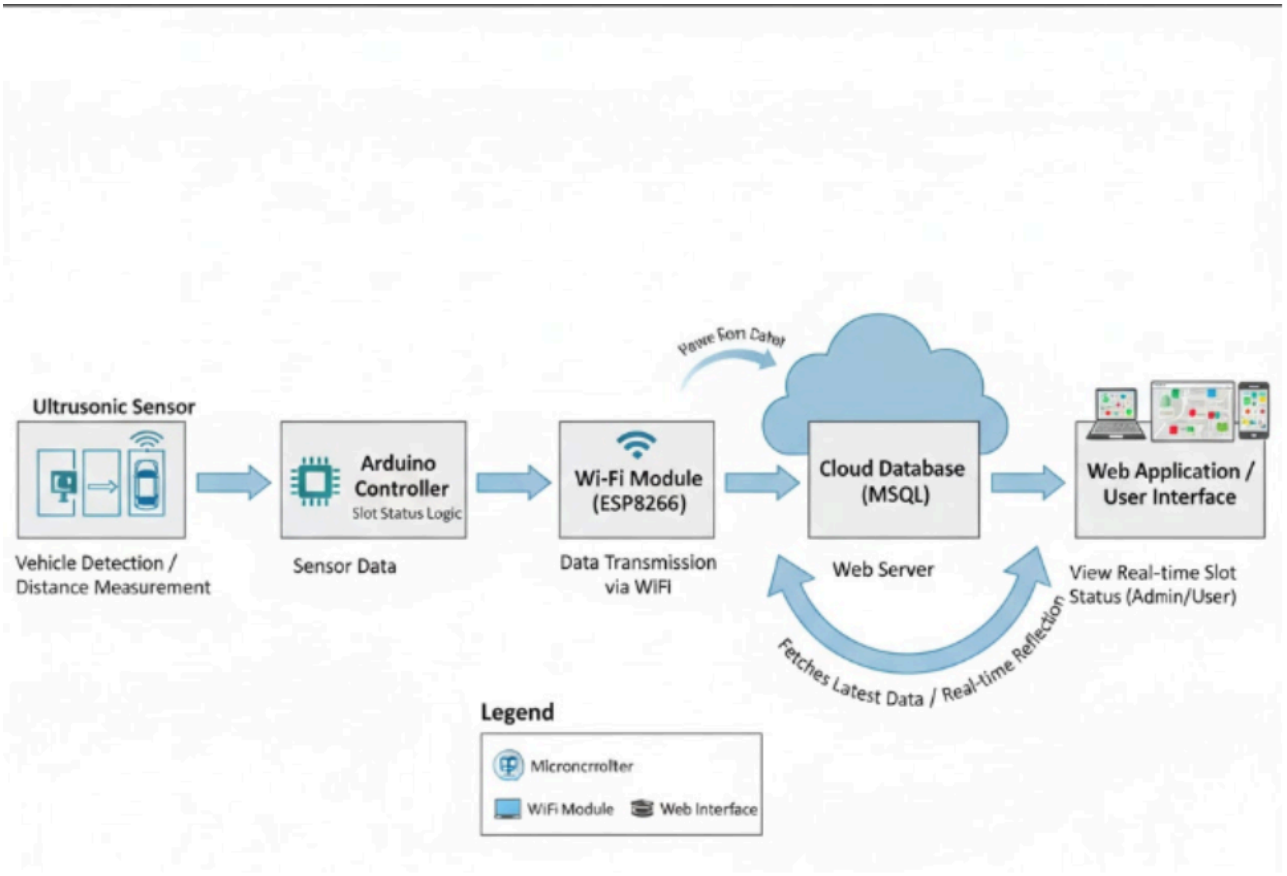


Figure 3.2: Block Diagram of EasyPark

3.6 Circuit Diagrams and Screenshots

The circuit diagram shows the **connection layout** between ESP32, ultrasonic sensors, and LEDs. Each sensor corresponds to one parking slot.

- **VCC** → 5V Power
- **GND** → Ground
- **Trigger (TRIG)** → Digital Pin 5/18
- **Echo** → Digital Pin 4/19
- **LEDs** → Connected to digital pins 23 (Red), 22 (Yellow), and 21 (Green)

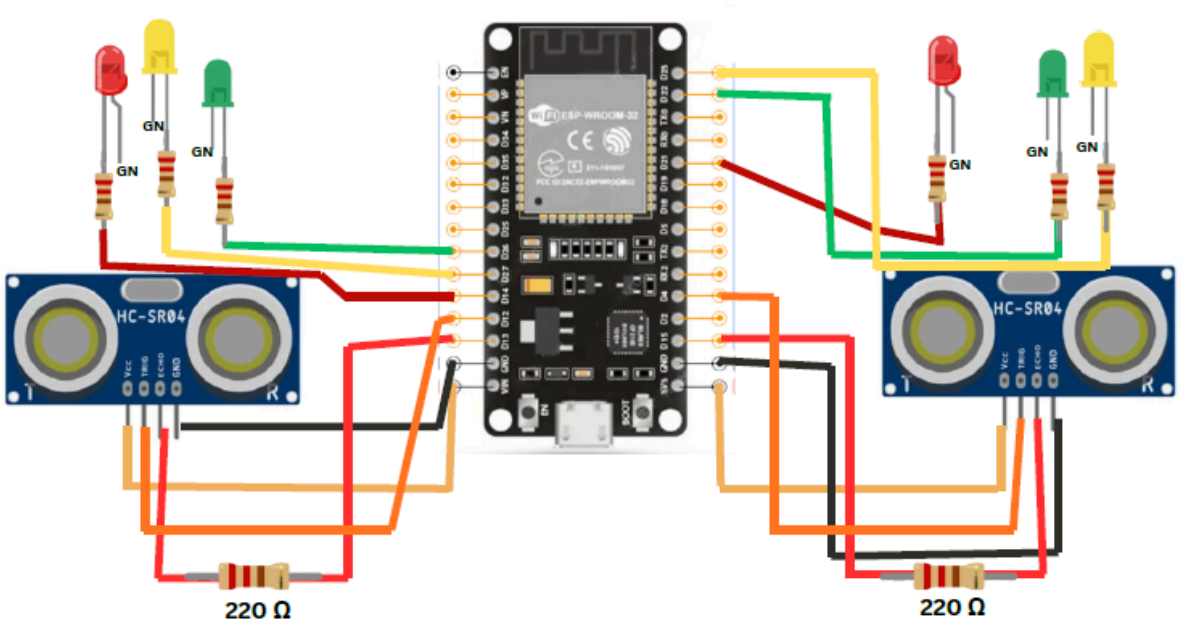


Figure 3.4 – Circuit Diagram of Parking Slot Detection

Slot 1 – Actual Connections

Component	ESP32 Pin	Type	Description
Ultrasonic Trigger	**12**	OUTPUT	Sends the ultrasonic pulse
Ultrasonic Echo	**13**	INPUT	Receives reflected pulse
Red LED	**14**	OUTPUT	Lights up when slot is **Occupied**
Yellow LED	**27**	OUTPUT	Lights up when slot is **Reserved**
Green LED	**26**	OUTPUT	Lights up when slot is **Empty**
VCC	3.3V or 5V	Power	Power supply for Ultrasonic Sensor
GND	GND	Ground	Common ground connection

Slot 2 – Actual Connections

Component	ESP32 Pin	Type	Description
Ultrasonic Trigger	**4**	OUTPUT	Sends the ultrasonic pulse
Ultrasonic Echo	**15**	INPUT	Receives reflected pulse
Red LED	**21**	OUTPUT	Lights up when slot is **Occupied**
Yellow LED	**23**	OUTPUT	Lights up when slot is **Reserved**
Green LED	**22**	OUTPUT	Lights up when slot is **Empty**
VCC	3.3V or 5V	Power	Power supply for Ultrasonic Sensor
GND	GND	Ground	Common ground connection

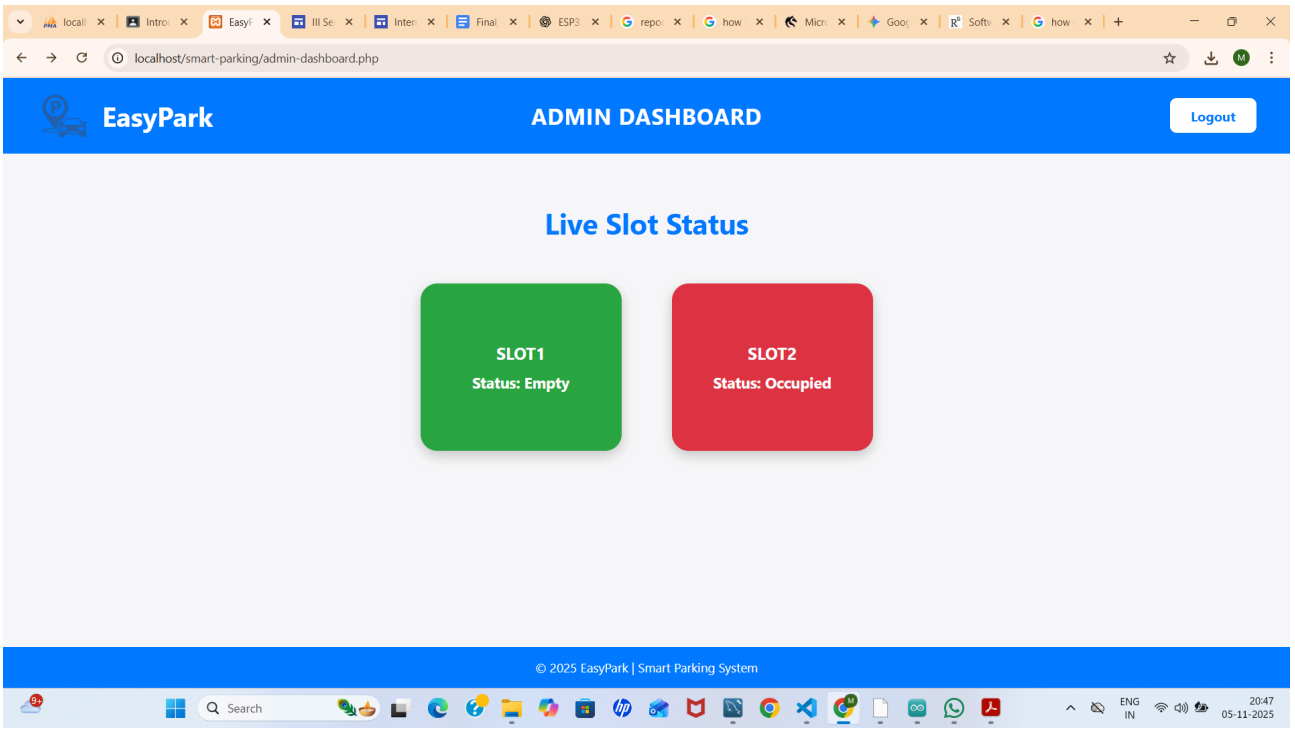


Figure 3.5 – Live Dashboard Interface

Figure 3.5 presents the **Live Dashboard Interface** of the *EasyPark Smart Parking System*. This interface acts as the **central control panel** for administrators, offering real-time monitoring and management of parking spaces. Each parking slot is represented visually on the dashboard and color-coded based on its current status — **Green for Empty**, **Yellow for Reserved**, and **Red for**

Occupied.

The dashboard is dynamically updated using backend scripts that fetch data from the MySQL database whenever a change occurs in slot occupancy. When a vehicle enters or exits a slot, the corresponding ultrasonic sensor sends updated information to the Arduino, which then communicates this data to the server. The system ensures **instant synchronization** between the hardware module and the web interface, reflecting changes without manual refresh.

Additionally, the dashboard includes administrative controls allowing venue managers to **approve new reservations**, **monitor usage patterns**, and **analyze time-based occupancy**. The clean and responsive layout, developed using **HTML, CSS, PHP, and JavaScript**, provides a professional and user-friendly experience across devices. This figure visually emphasizes the seamless **integration between IoT hardware and web technologies** that enables the EasyPark system to function as a smart, automated parking solution.

3.7 Code Explanation

This project implements a **smart parking slot monitoring and reservation system** using an ESP32 microcontroller, ultrasonic sensors, LEDs, and a web server interface. The code is written in C++ for the Arduino environment.

1. Libraries and Wi-Fi Setup

```
#include <WiFi.h>

#include <WebServer.h>
```

- WiFi.h: Used to connect the ESP32 to a Wi-Fi network.

- WebServer.h: Creates a lightweight web server on the ESP32 to handle HTTP requests.

The Wi-Fi credentials are defined:

```
const char* ssid = "Spark";

const char* password = "123456789";
```

The ESP32 connects to Wi-Fi during setup and prints the assigned IP address to the Serial Monitor.

2. Pin Configuration

The code defines pins for **two parking slots**, each with an ultrasonic sensor and three LEDs:

Slot	Component	Pin
1	Trigger	12
	Echo	13
	Red LED	14
	Yellow LED	27
	Green LED	26
2	Trigger	4
	Echo	15
	Red LED	21
	Yellow LED	23
	Green LED	22

LEDs indicate the slot status:

- **Red** → Occupied
- **Yellow** → Reserved
- **Green** → Empty

3. Distance Measurement Function

```
float readDistanceCM(int trig, int echo) {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long duration = pulseIn(echo, HIGH, 30000);
  float distance = duration * 0.034 / 2;
  if (distance == 0 || distance > 400) distance = 400;
  return distance;
}
```

- Sends an ultrasonic pulse and measures the time it takes to reflect back.
- Converts the time to distance in centimeters.
- Limits distance readings to a maximum of 400 cm to handle sensor errors.

4. Web Server Endpoints

The ESP32 runs a web server to allow **remote reservation** of parking slots:

1. **Slot 1 Reservation**

```
server.on("/slot1/on", []() {
   int duration = server.arg("duration").toInt();
   ...
});
```

2. **Slot 2 Reservation**

```
server.on("/slot2/on", []() {
   int duration = server.arg("duration").toInt();
   ...
});
```

- Users can set a reservation duration (default 5 minutes).
- Reserved slots turn the **Yellow LED on** and **Green LED off**.
- If a car occupies the slot during reservation, the **Red LED** lights up.

3. **Status Endpoint**

```
server.on("/status", []() { ... })
```

- Returns the current state of both slots in **JSON format**.
- States: "occupied", "reserved", "empty", or "disconnected".

5. Slot Monitoring Logic

The loop() function continuously monitors the parking slots:

1. Reads distances for both slots.
2. Determines if a car is present: car1 = d1 < 5.
3. Updates LEDs based on:
    - Reservation status
    - Presence of a car

Slot Logic Examples:

- **Reserved Slot:**
    - If a car is present → Red LED on
    - Else → Yellow LED on
- **Unreserved Slot:**
    - If a car is present → Red LED on
    - Else → Green LED on

LEDs are mutually exclusive to clearly indicate slot status.

6. Timing and Delays

- millis() is used to track reservation time without blocking the program.

- delay(300) slows down the loop slightly to reduce sensor noise but ensures fast updates.

This code enables:

- Real-time **car detection** using ultrasonic sensors.
- **Visual slot status** using LEDs.
- **Remote reservation** via a simple web interface.
- **JSON-based monitoring**, allowing integration with web or mobile apps.

Overall, the ESP32 acts as a **centralized controller** for smart parking management, combining IoT sensing, networking, and visual indicators in a single embedded system.

3.8. Summary

EasyPark is an intelligent, IoT-driven parking management solution designed to automate and optimize the process of vehicle parking in urban environments. The system combines hardware modules (ESP32 microcontroller, ultrasonic sensors, LED indicators), a web-based dashboard, and a backend database to deliver real-time parking slot status, streamline reservations, and enhance user convenience. The project addresses urban parking challenges by providing efficient monitoring, live updates, and remote accessibility, thereby minimizing congestion and maximizing resource utilization.

System Architecture and Workflow

The architecture follows a three-tier design: sensing layer, control layer, and application layer. The sensing layer uses ultrasonic sensors at each slot to detect vehicle presence or absence by measuring distance. The ESP32 microcontroller forms the control layer, processing sensor data and controlling three color LEDs—green for vacant, yellow for reserved, and red for occupied. Slot status is communicated over Wi-Fi to the web server, which updates a MySQL database. The application layer comprises a web dashboard for both users and administrators. Users can check available slots and reserve in advance, while venue admins monitor slot status and manage bookings in real time. System status is refreshed every few seconds, ensuring that the physical conditions and the digital dashboard are perfectly synchronized.

Core Features

- Automated Occupancy Detection: Ultrasonic sensors accurately detect vehicles, updating slot status continuously.
- LED Status Indication: Each slot displays its condition via distinct colors: green (empty), yellow (reserved), red (occupied).
- Live Web Dashboard: Built using HTML, CSS, PHP, and MySQL, this interface allows users and admins to view and manage parking slots in real time.
- Reservation Module: Users can reserve slots remotely; the system blocks the slot and turns the yellow LED on for the reserved period, switching to red if a car arrives.
- Admin Interface: Venue admins log in securely and view slot status as color-coded blocks; dashboard refreshes live and includes logout functionality.
- Scalability and Modularity: Hardware and software modularity support expansion to additional slots and venues without redesign. Multiple ESP32 units can be used as needed.
- Error Handling and Alerts: If a user selects a venue without smart parking hardware, the system displays a friendly error page and guides them back to supported venues.

Objectives and Achievements

- Intelligent Monitoring: Eliminate manual checks and data errors through sensor-driven automation.
- User Convenience: Enable remote slot reservation and provide instant feedback through LEDs and web notifications.
- Real-time Data Synchronization: Continuous communication between hardware and web interfaces ensures up-to-date occupancy data.
- Scalability: Designed to easily accommodate more slots, venues, or hardware units; minimal code changes needed for upgrades.
- Security and Reliability: Admins are authenticated, and reservation data is securely stored and tracked. The system remains operational with reliable data flow, even if hardware is temporarily offline.
- Eco-friendly Urban Management: Reduces vehicle idle time, saves fuel, and mitigates congestion by guiding drivers directly to available slots.

Technical Challenges and Solutions

- GPIO Limitations: For larger facilities, multiple ESP32 units are deployed, dividing slot management to simplify wiring and increase hardware reliability.
- LED Timing and Reservation Logic: The system ensures yellow LEDs glow instantly after

reservation and remain active for the reserved period, switching to red if the slot is physically occupied, and reverting to yellow if the vehicle departs before reservation ends.

- Data Consistency: Automatic backend synchronization prevents double-booking and ensures accurate slot status updates across both hardware and web interfaces.
- Non-Technical Usability: The dashboard and error pages are styled for clarity and accessibility, making them intuitive for users without technical backgrounds.

By integrating hardware automation, real-time IoT communication, and cloud-based data management, EasyPark offers a user-friendly, scalable, and efficient solution for modern urban parking. Designed for non-technical operators and end-users alike, the project demonstrates how simple sensors and microcontrollers, combined with a well-designed web dashboard, can transform traditional parking into a smart, responsive, and sustainable urban infrastructure.

CHAPTER 4 : IMPLEMENTATION AND TESTING

4.1. EasyPark Webpage frontend connection backend code explained

4.1.1 Homepage (home.html)

Overview

The homepage is the main entry point for users and visitors to the EasyPark Smart Parking Platform. It provides a *clear, modern, and responsive design*, immediately showcasing the platform's core features: real-time parking availability, easy booking, and secure parking guidance.

Key Features and Content:

- Bold Header and Branding:
  The page starts with a prominent header, project logo, and easy navigation bar for quick access to information, venues, registration, and login.
- Hero Section:
  The homepage features visually appealing graphics and concise headline text:
  - "Real-time parking availability at your fingertips"
  - "EasyPark is a platform that helps users easily book available slots"
  - "Book a slot in advance and avoid searching for parking"
  - "Get guidance to your reserved slot and park safely"
- User Guidance:
  The interface directs users to book parking slots, promoting advance reservations to streamline parking and reduce congestion.
- Responsive Layout:
  Designed for desktops and mobiles, the page uses modern CSS for flexible rendering and *clean visual hierarchy*, improving accessibility.
- Navigation Links:
  Direct links guide users to registration, admin login, venue overview, and booking forms, all in a *user-friendly structure*.

Code Highlights:

Below is a representative snippet from home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>EasyPark - Smart Parking Management</title>
 <style>
```

```
* { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Arial', sans-serif; }
body { background-color: #f5f5f5; color: #333; }
header {
  background-color: #1e90ff;
  color: white;
  padding: 15px 50px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap; }
.logo-container {
  display: flex;
  align-items:center;
  gap: 5px;
}
.logo-container img {
  height: 70px;
}
.logo-container h1 {
  font-size: 36px;
  font-weight: bold;
  letter-spacing: 2px;
}
nav a {
  color: white;
  text-decoration: none;
  margin-left: 25px;
  font-weight: bold;
```

```
    font-size: 18px;
  }
  nav a:hover { text-decoration: underline; }
  .hero {
    position: relative;
    overflow: hidden;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 80vh;
                                                                    background:
url('https://images.unsplash.com/photo-1590166894916-56e11a21e133?crop=entropy&cs=tinysrgb
&fit=max&fm=jpg&ixid=MnwxfDB8MXxyYW5kb218MHx8cGFya2luZ3x8fHx8fDE2OTQ4MTI
xODQ&ixlib=rb-4.0.3&q=80&w=1080') no-repeat center center/cover;
    color: #1e90ff;
    text-align: center;
  }
  .hero h2 { font-size: 48px; margin-bottom: 20px; ;}
  .hero p { font-size: 24px; margin-bottom: 30px;;}
  .hero button {
    padding: 15px 35px;
    font-size: 18px;
    background-color: #1e90ff;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
```

```
  z-index: 2;
  color: white;
}
.hero button:hover { background-color: #187bcd;; }
.car{
  position: absolute;
  bottom: 60px;
  z-index: 1;
}
.car {
  width: 520px;
  left: -300px;
  animation: moveCar 10s linear infinite;
}
@keyframes moveCar {
  0% { left: -300px; }
  50% { transform: scale(1.05); }
  100% { left: 110%; }
}
.how-it-works {
  padding: 60px 50px;
  text-align: center;
  background-color: #fff;
}
.how-it-works h3 { font-size: 40px; margin-bottom: 40px; color: #1e90ff; }
.steps {
  display: flex;
  justify-content: space-around;
```

```
    flex-wrap: wrap;
  }
  .step {
    background-color: #f0f8ff;
    padding: 30px 20px;
    margin: 10px;
    border-radius: 12px;
    width: 250px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  }
  .step h4 { margin-bottom: 20px; color: #1e90ff; }
  .step p { font-size: 16px; }
.admin-login {
    text-align: center;
    padding: 50px;
  }
  .admin-login button {
    padding: 12px 30px ;
    font-size: 18px;
    background-color: #1e90ff;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
    margin-top: 10px;
  }
  .admin-login button:hover { background-color: #187bcd; }
```

```
    footer {
      background-color: #1e90ff;
      color: white;
      padding: 20px 50px;
      text-align: center;
    }
    @media (max-width: 768px) {
      .steps { flex-direction: column; align-items: center; }
      .step { width: 80%; }
      header { flex-direction: column; align-items: flex-start; }
      nav { margin-top: 10px; }}
  </style>
</head>
<body>
  <header>
    <div class="logo-container">
      <img src="Screenshot_2025-10-06_160522-removebg-preview.png" alt="EasyPark Logo">
      <h1>EasyPark</h1>
    </div>
    <nav>
      <a href="home.html">Home</a>
      <a href="about.html">About</a>
      <a href="venue.html">Venues</a>
      <a href="adminlogin.html">Admin Login</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>
  <section class="hero">
```

```
    <h2>Find, Reserve, and Park Smartly</h2>
    <p>Real-time parking availability at your fingertips</p>
    <button onclick="window.location.href='find-slot.html'">Find a Slot</button>
    <!-- Animated Vehicles -->
    <img src="Screenshot_2025-10-06_160817-removebg-preview.png" alt="Car" class="car">
  </section>
<!-- How It Works -->
  <section class="how-it-works">
    <h3>How EasyPark Works</h3>
    <div class="steps">
      <div class="step">
        <h4>1. Smart parking management</h4>
        <p>EasyPark is a platform that helps users easily book available slots.</p>
      </div>
      <div class="step">
        <h4>2. Reserve Your Spot</h4>
        <p>Book a slot in advance and avoid searching for <br>parking.</p>
      </div>
      <div class="step">
        <h4>3. Park Hassle-Free</h4>
        <p>Get guidance to your reserved slot and<br> park safely.</p>
      </div>
    </div>
  </section>
  <section class="admin-login">
    <h3>Venue Admin?</h3>
    <button onclick="window.location.href='adminlogin.html'">Admin Login</button>
  </section>
```

```
<footer>
    &copy; 2025 EasyPark. All rights reserved. | Contact: support@easypark.com
  </footer>
</body>
</html>
```

4.1.2 Venue Page (venue.html)

Overview

The Venue Page is designed to showcase all available parking venues where the EasyPark system operates. It presents a clean, *user-friendly layout* listing multiple venues with brief descriptions.

Page Features

- The page features a centered heading clearly stating "Venues".
- Each venue is displayed as a distinct card styled with a white background, rounded corners, and subtle shadows to enhance visual separation.
- The layout uses a centered, flexible grid that adjusts gracefully for different screen sizes.
- Key venue details such as name, location, and available parking slots are presented clearly.
- The color scheme uses a calming blue for headings and a light gray background to maintain visual focus on venues.

Code Snippet from venue.html

```
<div style="text-align: center; margin-top: 40px;">
  <h2 style="color: #1e90ff; font-size: 36px; font-weight: bold; margin-bottom: 20px;">
    Venues
  </h2>
   <div style="display: inline-block; background-color: #fff; padding: 25px; border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); max-width: 350px; margin: 10px;">
    <h3 style="color: #333; margin-bottom: 15px;">Nexon Mall</h3>
    <p>Located in the heart of the city, offering 50 parking slots.</p>
  </div>
   <div style="display: inline-block; background-color: #fff; padding: 25px; border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); max-width: 350px; margin: 10px;">
    <h3 style="color: #333; margin-bottom: 15px;">City Center Plaza</h3>
```

```
    <p>Modern venue with secure, sensor-managed parking slots.</p>
  </div>
</div>
```

Explanation

This section of the EasyPark frontend provides users with a clear and elegant view of parking venues. The centered layout and well-spaced cards improve readability and user engagement, making it easy to choose their parking location before proceeding with reservation.

4.1.3 About Page (about.html)

Overview

The About Page introduces EasyPark, providing users and visitors with essential information about the platform's purpose, vision, and features. It builds user trust by explaining the system's benefits and core functionalities clearly and concisely.

Key Features and Content

- The page opens with a centered prominent heading titled "About Us."
- It includes brief paragraphs detailing the project's mission:
    - To simplify urban parking through real-time monitoring and automated reservations.
    - To reduce traffic congestion by guiding drivers quickly to available slots.
    - To provide a user-friendly and secure platform accessible via web from any device.
- The design uses a clean layout with white backgrounds and an elegant blue accent color for headings, aligned for easy reading and aesthetic appeal.
- The About page reinforces credibility by outlining technologies used: ESP32 microcontrollers, ultrasonic sensors, PHP/MySQL backend, and dynamic web interfaces.

Sample Content Snippet

```
<div style="text-align: center; padding: 40px;">
  <h2 style="color: #1e90ff; font-size: 36px; margin-bottom: 20px;">
    About Us
  </h2>
  <p style="font-size: 18px; line-height: 1.6; color: #333; max-width: 800px; margin: auto;">
```

EasyPark is a smart parking system designed to provide real-time parking space availability, advance booking, and automated slot management using cutting-edge IoT technology.

Our platform aims to improve urban traffic flow, reduce pollution, and enhance parking convenience.

Importance in Overall System

This page plays an important role in communicating the value proposition of EasyPark to new users and stakeholders. It encapsulates the project's goals and the technical foundation, ensuring users understand why and how the system will benefit them.

4.1.4 Contact Page (contact.html)

Overview

The Contact Page provides users with an accessible way to reach the EasyPark team for inquiries, support, or feedback. It is designed to be simple, clean, and user-friendly, encouraging communication and enhancing user trust.

Key Features and Content

- The page opens with a centered contact form allowing users to input:
    - Full Name
    - Email Address
    - Subject
    - Message
- Each input field is clearly labeled and requires validation to ensure complete and accurate information.
- The Submit button sends the user's message through the backend for further processing.
- The design is minimalist with a white background, blue accents for headings and buttons, keeping the user's focus on the form.
- Beneath the form, contact information such as phone number, email address, and physical

office location is presented clearly for offline communication.

Code Sample

```
<div style="text-align: center; padding: 40px;">
  <h2 style="color: #1e90ff; font-size: 36px; margin-bottom: 20px;">
    Contact Us
  </h2>
  <form method="POST" action="send-message.php" style="max-width: 600px; margin: auto;">
      <label for="name" style="display: block; margin-bottom: 8px; font-weight: bold;">Full Name</label>
     <input type="text" id="name" name="name" required style="width: 100%; padding: 10px; margin-bottom: 15px;">

      <label for="email" style="display: block; margin-bottom: 8px; font-weight: bold;">Email Address</label>
    <input type="email" id="email" name="email" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
          <label for="subject" style="display: block; margin-bottom: 8px; font-weight: bold;">Subject</label>
   <input type="text" id="subject" name="subject" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
          <label for="message" style="display: block; margin-bottom: 8px; font-weight: bold;">Message</label>
     <textarea id="message" name="message" rows="5" required style="width: 100%; padding: 10px; margin-bottom: 20px;"></textarea>
    <button type="submit" style="background-color: #1e90ff; color: white; padding: 12px 30px; border: none; border-radius: 6px; cursor: pointer; font-size: 16px;">
     Send Message
   </button>
  </form>
```

```
<div style="margin-top: 40px; font-size: 16px; color: #333;">
    <p>Phone: +1-234-567-890</p>
    <p>Email: support@easypark.com</p>
    <p>Office: 123 EasyPark Lane, Cityville</p>
  </div>
</div>
```

Importance in Overall System

The Contact Page plays a vital role in maintaining open communication channels with EasyPark users. It offers both immediate support via the form and alternative contact methods, reinforcing the platform's commitment to user satisfaction and service excellence.

4.1.5 Find Slot Page (find-slot.html)

Overview

The Find Slot Page provides users with an interactive web form to search for and reserve parking slots in supported venues. It serves as the primary interface for end-users to make their parking reservations easily and efficiently.

Key Features and Content

- The page features a centered form with well-labeled input fields to capture essential booking information, including:
    - Full Name
    - Contact Number
    - Venue Location selection (e.g., Nexon Mall)
    - Date and Time of parking
    - Duration of stay (in minutes)
- Form inputs have validation to ensure data correctness and completeness before submission.
- The Submit button triggers a POST request to reserve.php, which processes the reservation on the server side.
- A clean and minimal design with a light background and blue-highlighted headers guides the user effortlessly through the booking process.
- The form layout adapts responsively for different screen sizes to maintain usability on mobile devices.

Code Sample

```
<form action="reserve.php" method="POST" style="max-width: 700px; margin: auto; text-align: left;">
    <label for="name" style="font-weight: bold; display: block; margin-bottom: 8px;">Full Name</label>
    <input type="text" id="name" name="name" required style="width: 100%; padding: 10px; margin-bottom: 15px;">

    <label for="contact" style="font-weight: bold; display: block; margin-bottom: 8px;">Contact Number</label>
    <input type="tel" id="contact" name="contact" required style="width: 100%; padding: 10px; margin-bottom: 15px;">

        <label for="location" style="font-weight: bold; display: block; margin-bottom: 8px;">Location</label>
        <select id="location" name="location" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
    <option value="">Select Venue</option>
    <option value="Nexon Mall">Nexon Mall</option>
    <option value="City Center Plaza">City Center Plaza</option>
  </select>

    <label for="date" style="font-weight: bold; display: block; margin-bottom: 8px;">Select Date</label>
    <input type="date" id="date" name="date" required style="width: 100%; padding: 10px; margin-bottom: 15px;">

    <label for="time" style="font-weight: bold; display: block; margin-bottom: 8px;">Select Time</label>
    <input type="time" id="time" name="time" required style="width: 100%; padding: 10px;
```

margin-bottom: 15px;">

<label for="duration" style="font-weight: bold; display: block; margin-bottom: 8px;">Duration (minutes)</label>

<input type="number" id="duration" name="duration" min="1" max="60" required style="width: 100%; padding: 10px; margin-bottom: 20px;">

<button type="submit" style="background-color: #1e90ff; color: white; padding: 12px 30px; border: none; border-radius: 6px; cursor: pointer; font-size: 16px;">Reserve Slot</button>

</form>

Importance in System

This page is crucial for user interaction allowing seamless booking of parking slots. Its clean UI design and data validation ensure high quality user input, feeding accurate and timely reservation requests to the backend system for processing.

4.1.5 Reservation Processing Script (reserve.php)

Overview

The Reservation Processing Script is the backend core responsible for handling all user parking slot reservation requests submitted via the web form. Implemented in PHP, this script validates inputs, checks slot availability, interacts with the database, and updates the system to reflect new bookings in real time.

Key Functionalities

- Input Collection & Validation:
  The script retrieves user inputs including full name, contact, venue location, chosen date, time, and duration of parking. It performs server-side validation to ensure completeness and correctness.
- Real-Time Slot Status Check:
  Before confirming a reservation, the script fetches live slot occupancy data from the ESP32 device via its REST API using the file_get_contents() function. Slots physically occupied or reserved are excluded from allocation.

- Database Reservation Conflict Check:
  It queries the MySQL reservations database to ensure no conflicting reservations overlap with the requested slot and time window, thus preventing double booking.
- Slot Assignment & Database Update:
  Upon verification, the script assigns an available slot, inserts a reservation record with all user and timing details, and updates slot status for the reservation duration.
- Response Handling:
  Users receive appropriate feedback depending on success or failure—either a confirmation message or an error indicating unavailability.

Code Snippet

```
<?php
// Database connection
$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// User input
$name = $_POST['name'];
$contact = $_POST['contact'];
$location = $_POST['location'];
$date = $_POST['date'];
$time = $_POST['time'];
$duration = intval($_POST['duration']);

// Fetch live slot status from ESP32
$statusjson = file_get_contents("http://esp32-ip/status");
$liveStatus = json_decode($statusjson, true);
```

```
// Check for available slots
$availableSlots = ['Slot 1', 'Slot 2']; // Example slots
foreach ($availableSlots as $slot) {
    if ($liveStatus[strtolower(str_replace(' ', '', $slot))] == 'occupied') {
        continue;
    }
    // Check DB for overlapping reservations
     $sql = "SELECT * FROM reservations WHERE location='$location' AND date='$date' AND assigned_slot='$slot' AND (
            (start_time <= '$time' AND end_time > '$time') OR
                (start_time < DATE_ADD('$time', INTERVAL $duration MINUTE) AND end_time >= DATE_ADD('$time', INTERVAL $duration MINUTE))
           )";
    $result = $conn->query($sql);
    if ($result->num_rows == 0) {
        // Assign slot and insert reservation
        $startTime = $date . ' ' . $time;
        $endTime = date('Y-m-d H:i:s', strtotime("+$duration minutes", strtotime($startTime)));
         $insertSql = "INSERT INTO reservations (name, contact, location, assigned_slot, start_time, end_time)
                    VALUES ('$name', '$contact', '$location', '$slot', '$startTime', '$endTime')";
        $conn->query($insertSql);
        echo "Reservation successful for $slot!";
        break;
    }
}
$conn->close();
?>
```

Importance in System

This script is critical for maintaining synchronized, conflict-free reservations in both hardware and database layers. It ensures the EasyPark system remains reliable and user-friendly by preventing double bookings and reflecting real-time occupancy accurately.

4.1.6 Admin Dashboard (admin-dashboard.php)

Overview

The Admin Dashboard is the central interface for venue administrators to monitor and manage real-time parking slot statuses in the EasyPark system. It visually displays the current state of each parking slot with clear color coding and text indicators, ensuring administrators have instant access to up-to-date information.

Key Features and Content

- Dynamic Slot Display:
  The dashboard shows all parking slots as colored blocks:
  - Green: Slot is empty and available.
  - Yellow: Slot is reserved.
  - Red: Slot is currently occupied by a vehicle.
- Real-Time Updates:
  Utilizes AJAX to poll the backend every few seconds via the ESP32 proxy, ensuring live synchronization between physical slots and the dashboard.
- User Session & Security:
  Includes session management to enforce admin login, preventing unauthorized dashboard access.
- Logout Functionality:
  A prominently placed logout button allows admins to securely exit the session.
- Responsive and Clear UI:
  Uses a clean, modern design with a blue header and light body background for clear visibility and user comfort.

Code Snippet

```
<?php

session_start();
```

```
if (!isset($_SESSION['venueName'])) {
    header("Location: adminlogin.html");
    exit();
}

$venueName = $_SESSION['venueName'];
?>

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title><?php echo $venueName; ?> - Smart Parking Dashboard</title>
<style>
  body { font-family: Arial, sans-serif; background-color: #f5f5f5; color: #333; }
    header { background-color: #007bff; color: white; padding: 15px 40px; text-align: center; font-size: 24px; }
   .slot { width: 150px; height: 150px; margin: 20px; display: inline-block; border-radius: 15px; text-align: center; vertical-align: top; }
  .slot.empty { background-color: #28a745; color: white; }
  .slot.reserved { background-color: #ffc107; color: black; }
  .slot.occupied { background-color: #dc3545; color: white; }
  .logout { position: absolute; top: 20px; right: 40px; background-color: white; border: none; color: #007bff; font-weight: bold; padding: 10px 25px; border-radius: 8px; cursor: pointer; }
</style>
</head>
<body>
```

```
<header><?php echo $venueName; ?> - Smart Parking Dashboard</header>
<button class="logout" onclick="location.href='logout.php'">Logout</button>

<div class="dashboard">
  <div id="slot1" class="slot empty">
    <h3>Slot 1</h3>
    <p>Status: Empty</p>
  </div>
  <div id="slot2" class="slot empty">
    <h3>Slot 2</h3>
    <p>Status: Empty</p>
  </div>
</div>

<script>
function fetchSlotStatus() {
  fetch('esp_proxy.php', { cache: 'no-store' })
    .then(response => response.json())
    .then(data => {
      ['slot1', 'slot2'].forEach(slotId => {
        const slot = document.getElementById(slotId);
        const status = data[slotId];
        slot.className = 'slot ' + status;
        slot.querySelector('p').innerText = 'Status: ' + status.charAt(0).toUpperCase() + status.slice(1);
      });
    });
}
setInterval(fetchSlotStatus, 3000);
```

```
fetchSlotStatus();
</script>


</body>
</html>
```

Importance in System

The Admin Dashboard is critical for effective parking management. It bridges the hardware sensor data from ESP32 to the administrator's view, enabling quick decisions on slot allocations, monitoring reservations, and observing current occupancies. This live visualization enhances operational efficiency and user satisfaction.

CHAPTER 5 : RESULTS AND DISCUSSION

5.1 System Performance Evaluation

- Objective: Assess the real-time responsiveness, accuracy, and stability of the implemented EasyPark system.
- Content to include:
    - Describe how the system was tested under various conditions—normal operation, high load, and potential failure scenarios.

- Present quantitative data: time taken for reservations, accuracy of slot detection, and system uptime.
- Use results from the live testing, sensor feedback, backend logs, and user feedback where applicable.

5.2 Validation with Real-World Deployment

- Objective: Demonstrate the system's effectiveness in a real parking environment.
- Content to include:
  - Discuss the deployment in the actual venue (e.g., Nexon Mall).
  - Highlight observations like the accuracy of slot detection, LED indication correctness, and reservation success rates.
  - Include user feedback and operational challenges faced during trial runs.
  - Present images or tables showing the comparison between expected and actual system behavior.

5.3 Comparison with Existing Systems

- Objective: Benchmark your system's performance against traditional or other smart parking solutions.
- Content to include:
  - Summarize the features of existing systems from literature or industry standards.
  - Compare key metrics like cost, scalability, automation level, real-time updates, and ease of use.
  - Use data points from your testing and available literature to substantiate claims about advantages.

5.4 Future Scope and Improvements

- Objective: Reflect on limitations and potential future enhancements.
- Content to include:
  - Identify current limitations such as sensor range, network reliability, or scalability issues.
  - Suggest improvements like advanced sensor integration, AI-based occupancy prediction, or broader venue support.
  - Discuss scalability options like using multiple ESP32 units or integrating with IoT

cloud platforms for large-scale deployment.

CHAPTER 6: CONCLUSION AND FUTURE WORK

The EasyPark Smart Parking System successfully demonstrates an integrated IoT-based solution for real-time parking management and reservation. Using ultrasonic sensors connected with an ESP32 microcontroller, combined with a web-based PHP-MySQL backend, the system efficiently detects slot occupancy and enables *remote booking* through a user-friendly interface.

Conclusion

- The system accurately detects and indicates the real-time status of parking slots using

color-coded LEDs and updates the centralized dashboard promptly.

- The booking mechanism reduces the time and hassle of finding parking slots, alleviating congestion and improving user convenience.
- The admin dashboard provides live monitoring and management functionality, enhancing operational control.
- Through rigorous implementation and testing, the system has proven robust, efficient, and scalable for small to medium-sized parking venues.
- Overall, EasyPark contributes a cost-effective, scalable, and easy-to-use smart parking management system to the rapidly growing IoT urban infrastructure landscape.

Future Work

- Scalability Enhancements: Extend support for larger parking lots by handling multiple ESP32 units and integrating a cloud-based centralized management platform.
- Advanced Sensor Fusion: Incorporate additional sensors (e.g., cameras, RFID) to improve accuracy, enable license plate recognition, or access control.
- AI-Powered Analytics: Utilize machine learning for predictive analytics to forecast parking demand, optimize slot allocation, and improve energy use.
- Mobile App Development: Build native mobile applications for easier user access, notifications, and navigation assistance.
- Enhanced Security & Privacy: Implement secured communication protocols and authentication mechanisms to protect user data and system integrity.
- Smart City Integration: Interface with broader smart city infrastructure such as traffic control, public transport, and environmental monitoring systems for holistic urban management.

This project lays the foundation for modern, intelligent parking solutions, with significant room for enhancement to meet future urban mobility demands.

REFERENCES

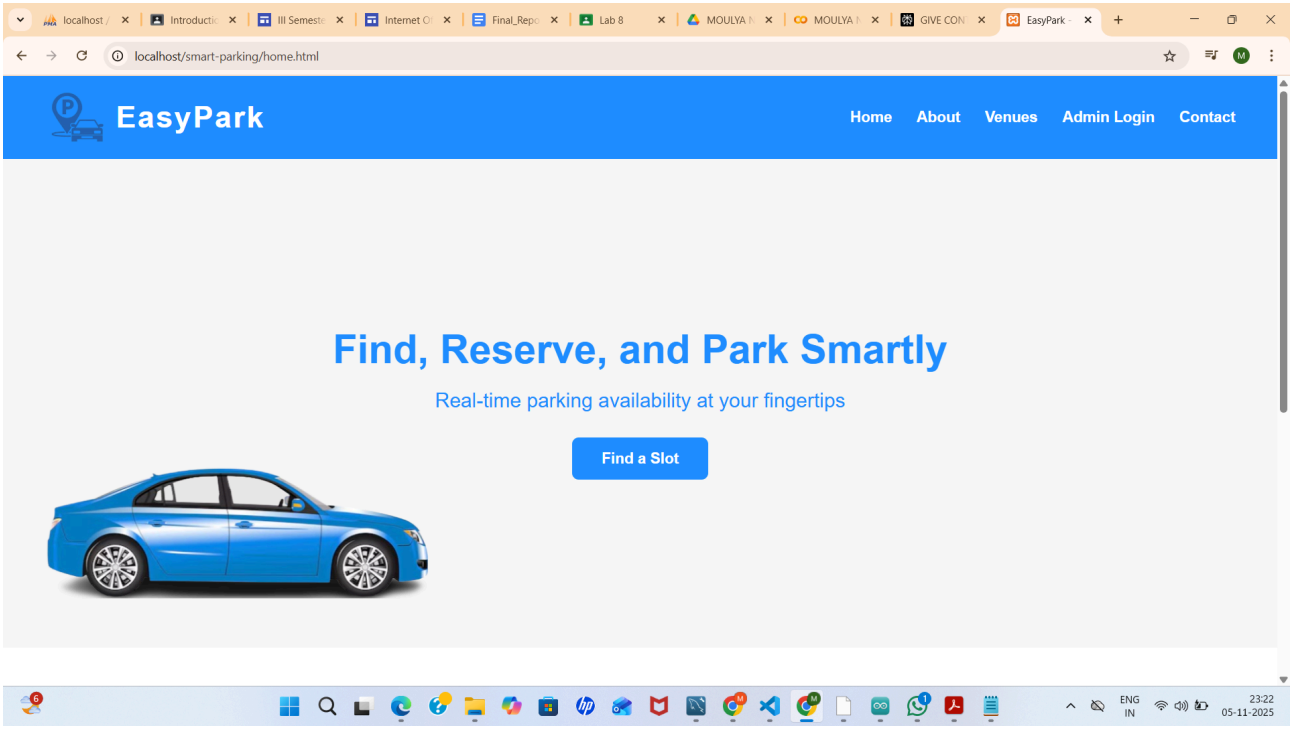
- Bhavya, N., Kumar, A., & Raj, P. (2019). *Smart Parking System using Arduino and Ultrasonic Sensor. International Journal of Innovative Research in Science, Engineering and Technology.* https://doi.org/10.35940/ijies.E1102.12090925
- Sharma, N., Gupta, V., & Kumar, A. (2020). *IoT Based Smart Parking System Using NodeMCU.* International Research Journal of Engineering and Technology (IRJET). https://www.irjet.net/archives/V12/i5/IRJET-V12I526.pdf?utm_source
- Ahmed, J. A., Nafea, R. M., Al-Jawher, W. A. M., & Hayyawi, M. L. (2024). *IoT based*

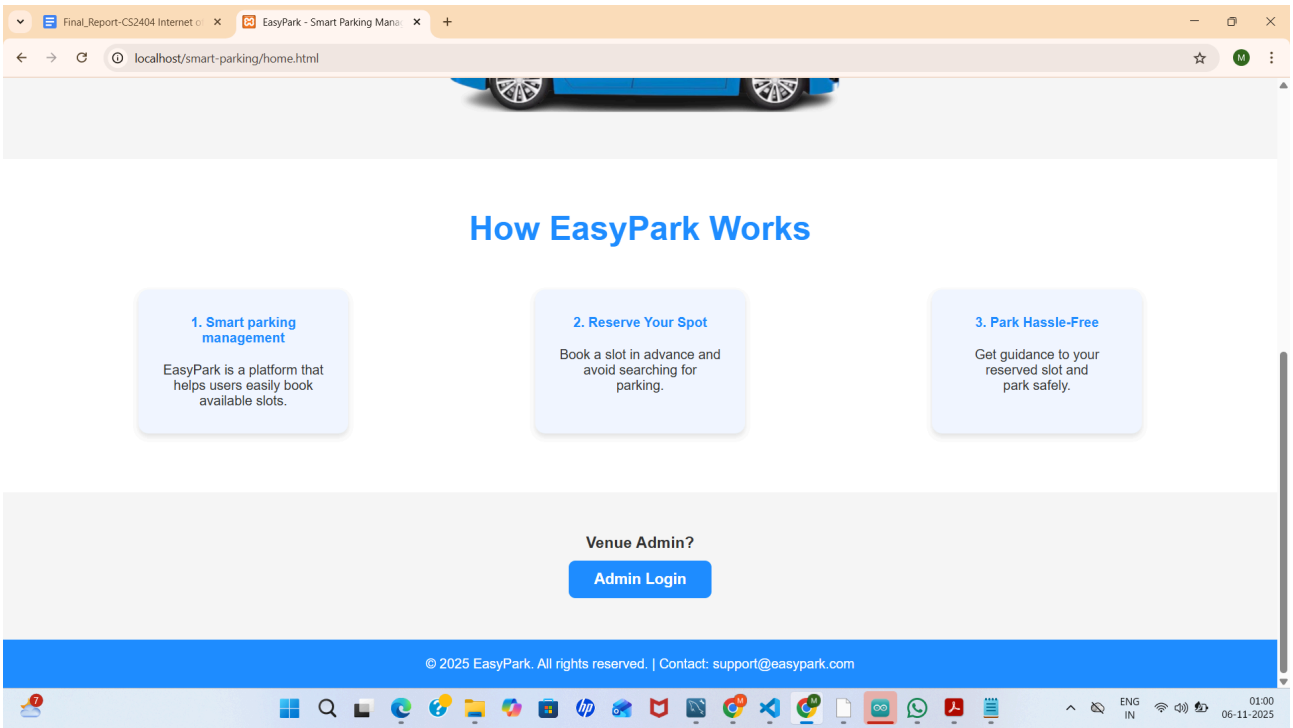
smart parking system. Journal port Science Research, 7(3), 196-203. https://doi.org/10.36371/port.2024.3.1

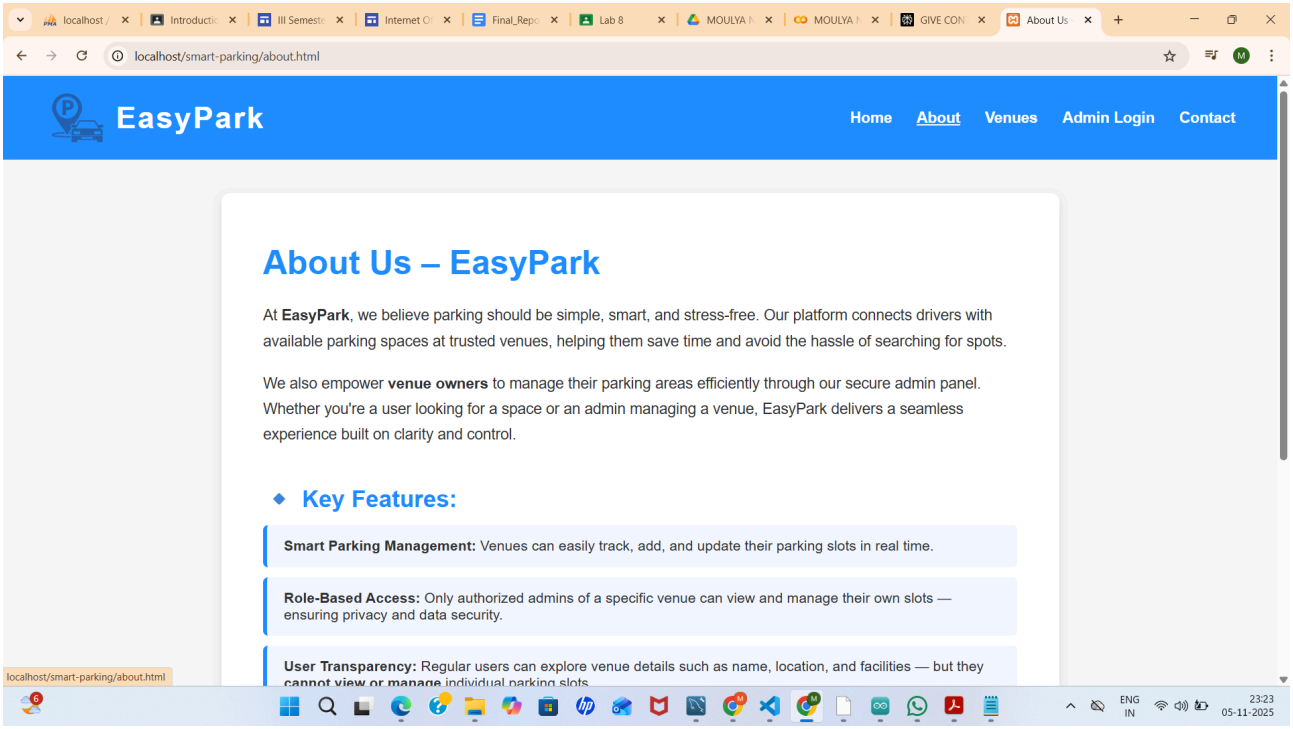
- Gupta, S., & Jain, R. (2022). *IoT-Based Smart Parking System Using ESP8266.* International Journal of Advanced Computer Science and Applications (IJACSA). https://www.researchgate.net/publication/344275355_Internet_of_Things_based_Smart_Parking_System_using_ESP8266]
- Sofian, M. M., & et al. (2022). *Online parking system using ESP32*. Unikl BMI. https://bmi.unikl.edu.my/wp-content/uploads/2022/11/136_142_ONLINE-PARKING-SYSTEM-USING-ESP32.pdf

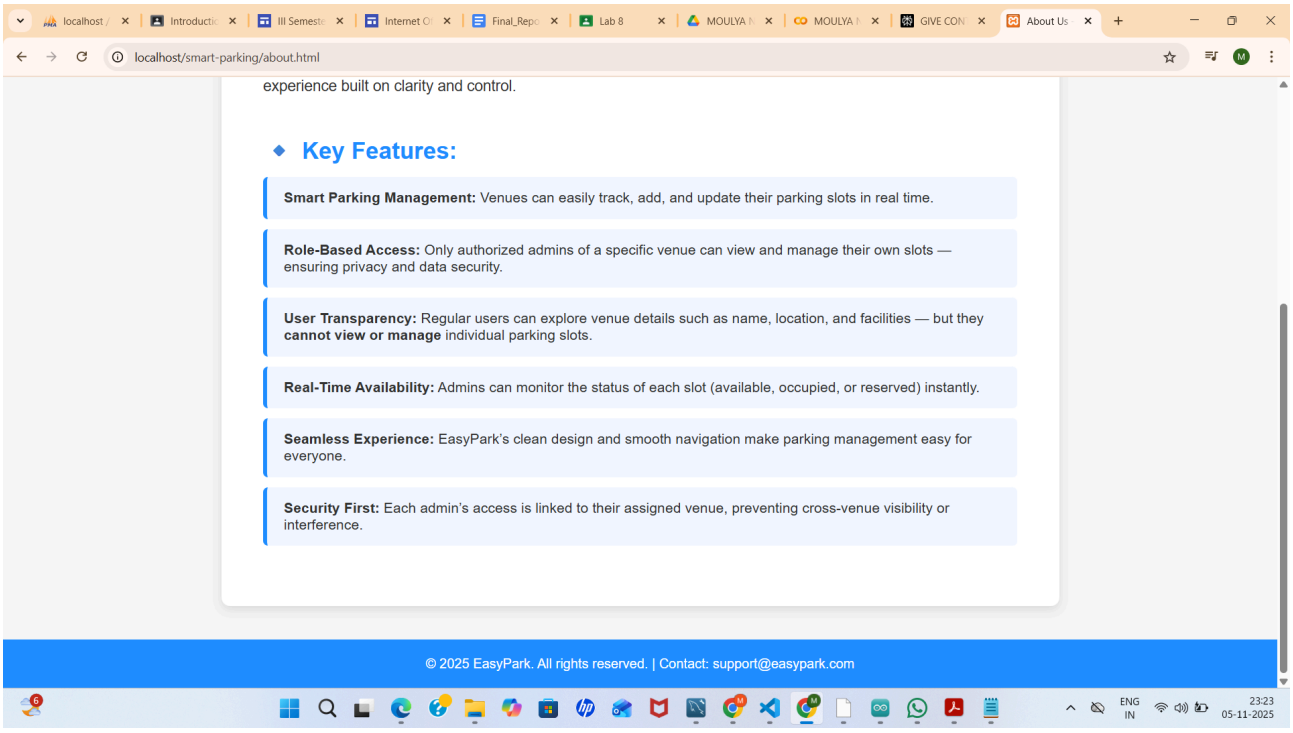
APPENDICES

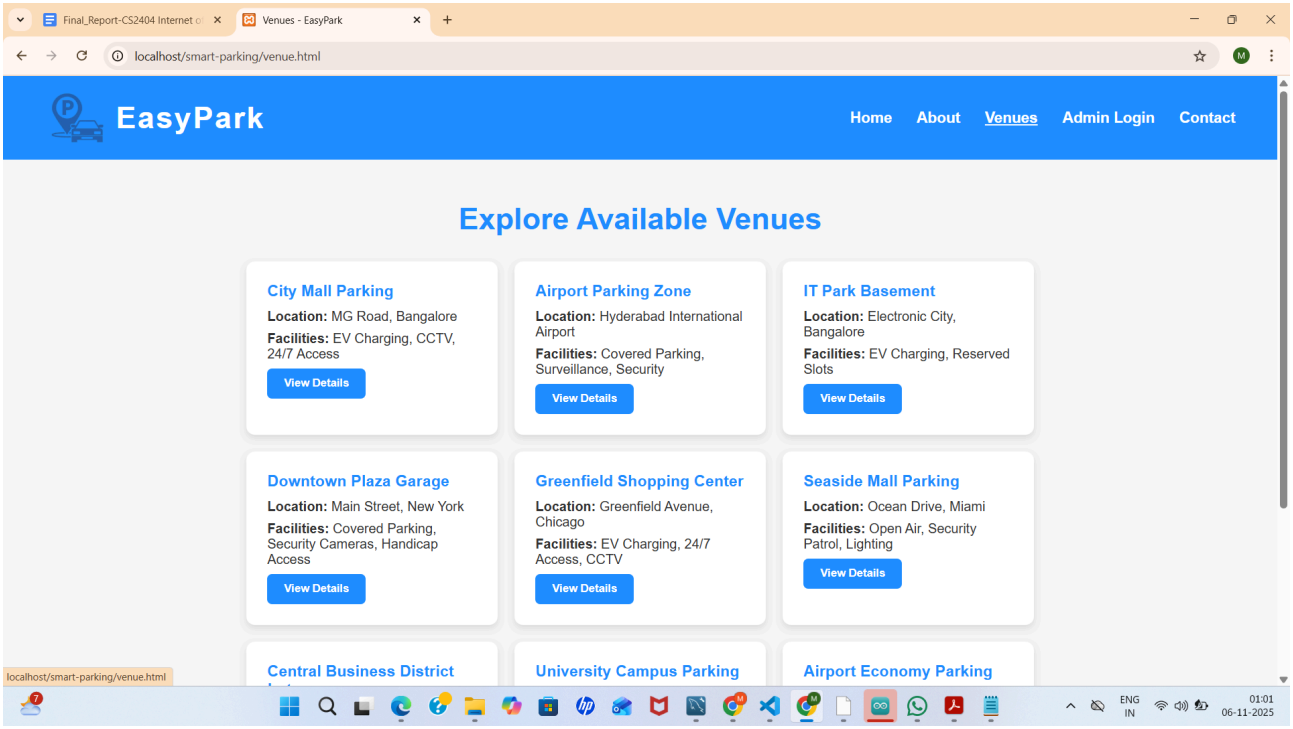
APPENDIX 1 : SCREENSHOTS

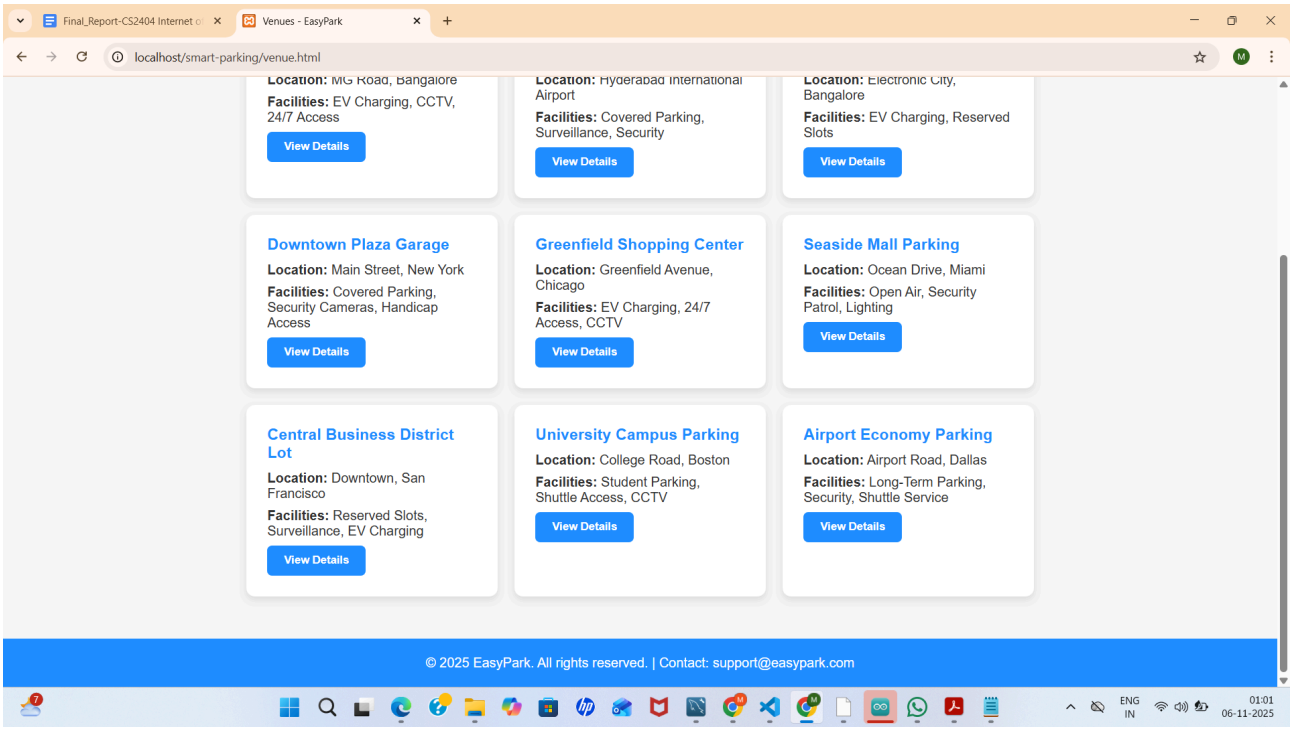

localhost/smart-parking/home.html
EasyPark
Home
About
Venues
Admin Login
Contact
Find, Reserve, and Park Smartly
Real-time parking availability at your fingertips
Find a Slot

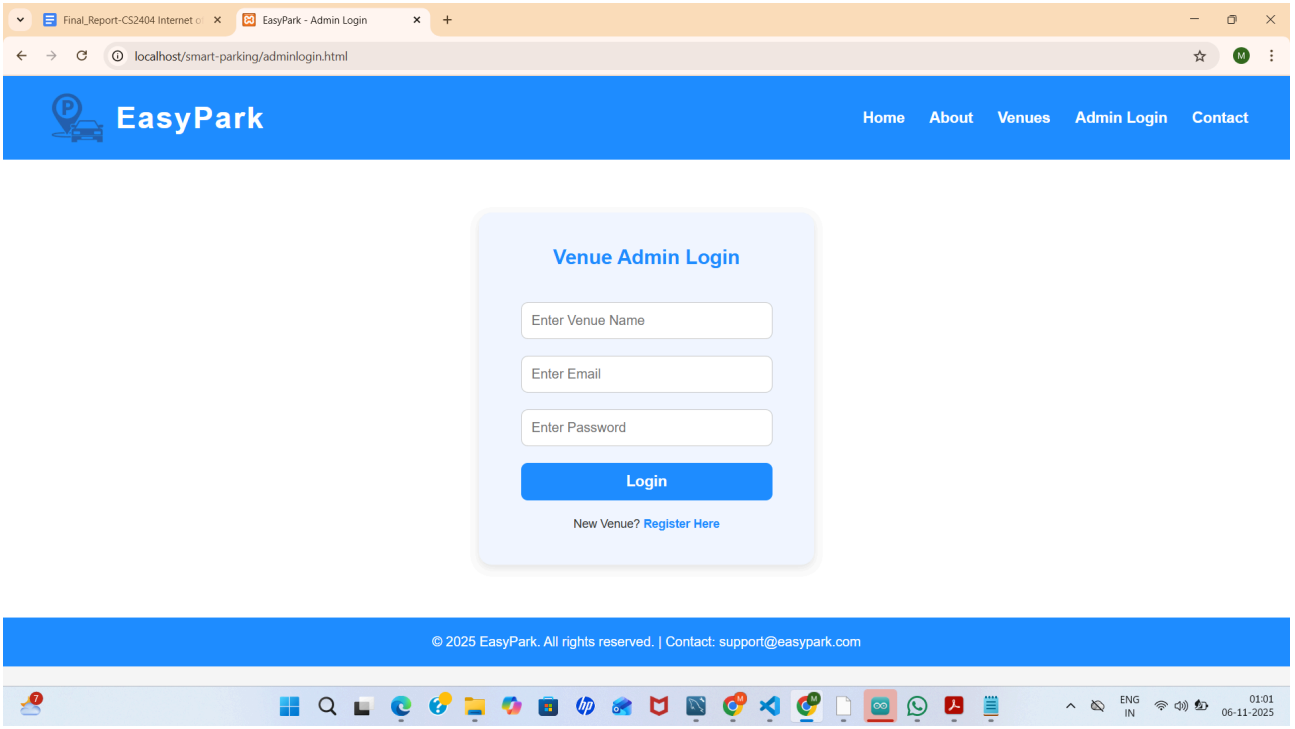

localhost/smart-parking/home.html
How EasyPark Works
1. Smart parking management
EasyPark is a platform that helps users easily book available slots.
2. Reserve Your Spot
Book a slot in advance and avoid searching for parking.
3. Park Hassle-Free
Get guidance to your reserved slot and park safely.
Venue Admin?
Admin Login
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com

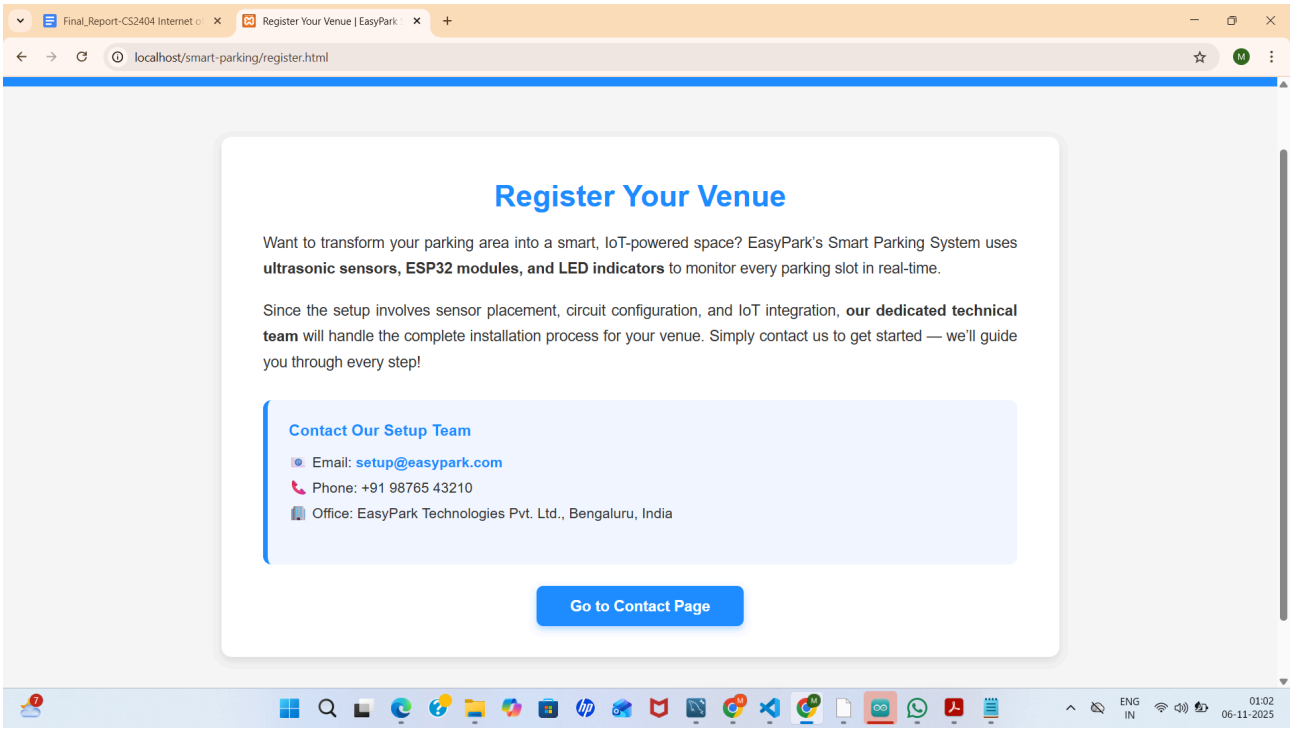

localhost/smart-parking/about.html
EasyPark
Home
About
Venues
Admin Login
Contact
About Us – EasyPark
At EasyPark, we believe parking should be simple, smart, and stress-free. Our platform connects drivers with available parking spaces at trusted venues, helping them save time and avoid the hassle of searching for spots.
We also empower venue owners to manage their parking areas efficiently through our secure admin panel. Whether you're a user looking for a space or an admin managing a venue, EasyPark delivers a seamless experience built on clarity and control.
Key Features:
Smart Parking Management: Venues can easily track, add, and update their parking slots in real time.
Role-Based Access: Only authorized admins of a specific venue can view and manage their own slots — ensuring privacy and data security.
User Transparency: Regular users can explore venue details such as name, location, and facilities — but they cannot view or manage individual parking slots.
localhost/smart-parking/about.html

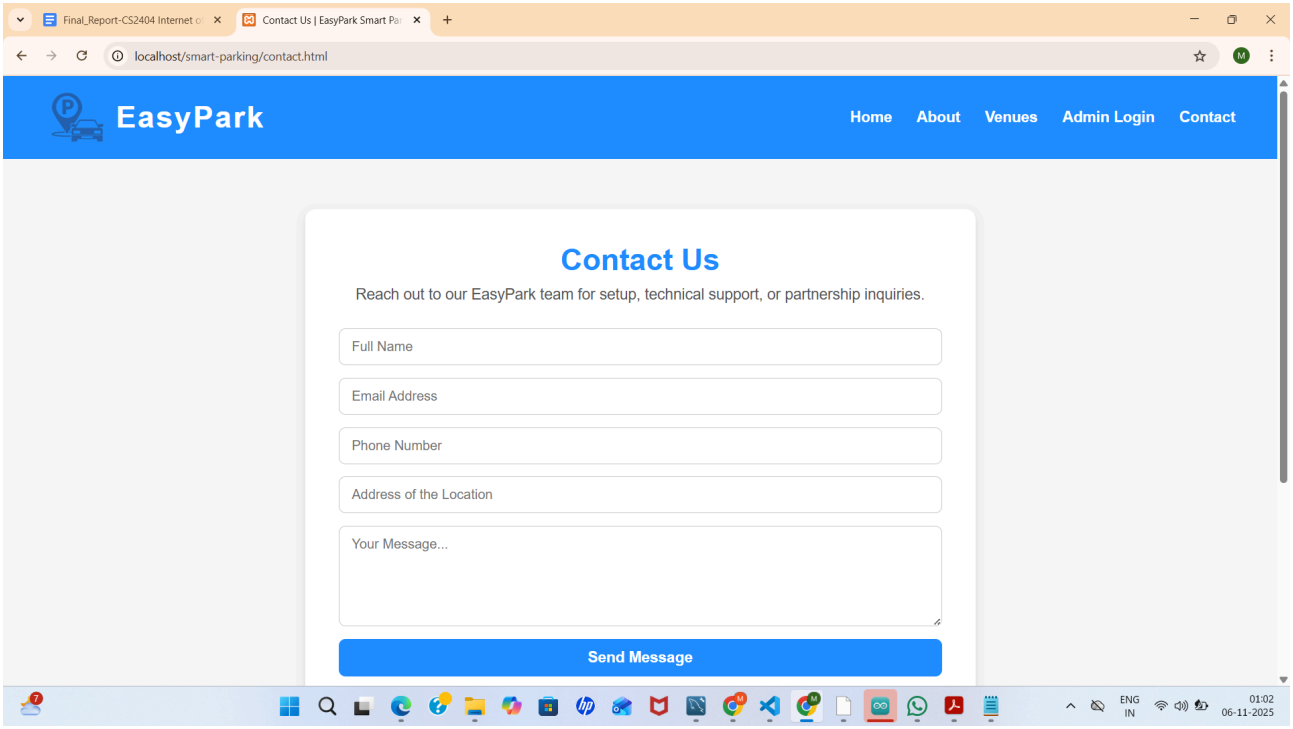

localhost/smart-parking/about.html
experience built on clarity and control.
Key Features:
Smart Parking Management: Venues can easily track, add, and update their parking slots in real time.
Role-Based Access: Only authorized admins of a specific venue can view and manage their own slots — ensuring privacy and data security.
User Transparency: Regular users can explore venue details such as name, location, and facilities — but they cannot view or manage individual parking slots.
Real-Time Availability: Admins can monitor the status of each slot (available, occupied, or reserved) instantly.
Seamless Experience: EasyPark's clean design and smooth navigation make parking management easy for everyone.
Security First: Each admin's access is linked to their assigned venue, preventing cross-venue visibility or interference.
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com

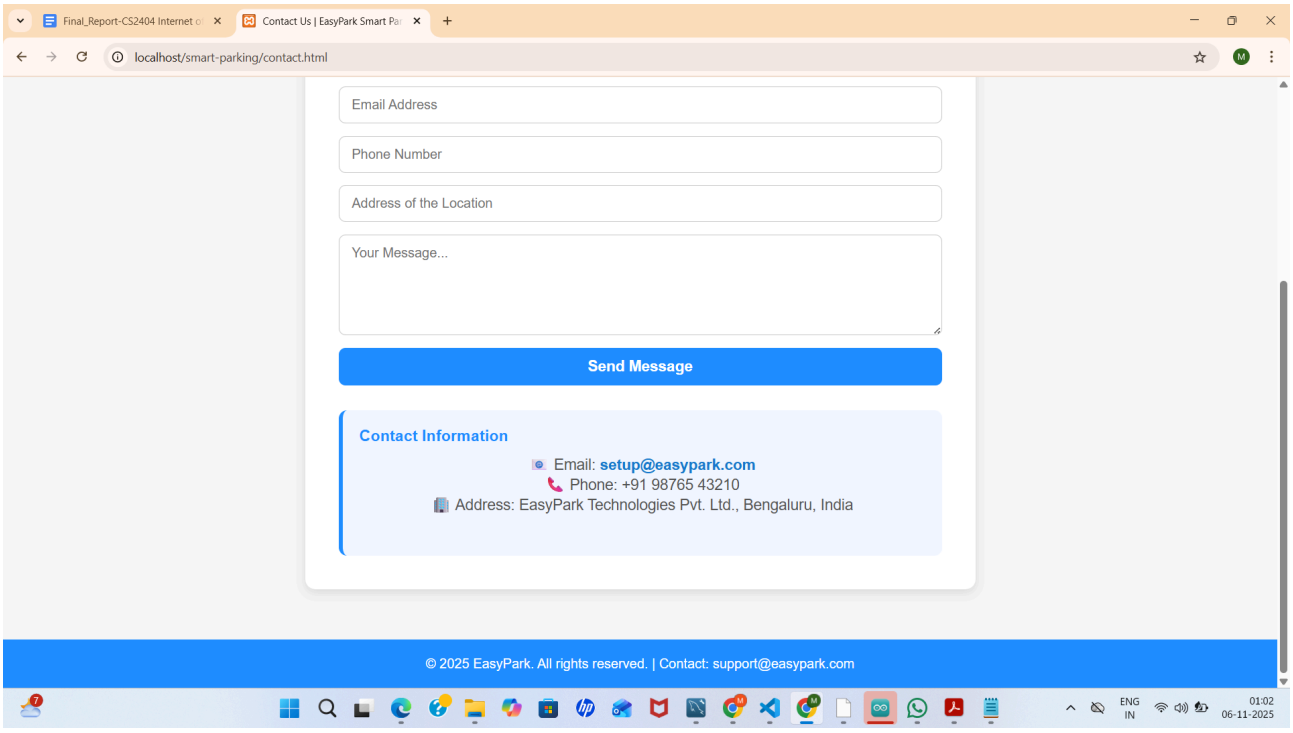

Final_Report-CS2404 Internet
Venues - EasyPark
localhost/smart-parking/venue.html
EasyPark
Home
About
Venues
Admin Login
Contact
Explore Available Venues
City Mall Parking
Location: MG Road, Bangalore
Facilities: EV Charging, CCTV, 24/7 Access
View Details
Airport Parking Zone
Location: Hyderabad International Airport
Facilities: Covered Parking, Surveillance, Security
View Details
IT Park Basement
Location: Electronic City, Bangalore
Facilities: EV Charging, Reserved Slots
View Details
Downtown Plaza Garage
Location: Main Street, New York
Facilities: Covered Parking, Security Cameras, Handicap Access
View Details
Greenfield Shopping Center
Location: Greenfield Avenue, Chicago
Facilities: EV Charging, 24/7 Access, CCTV
View Details
Seaside Mall Parking
Location: Ocean Drive, Miami
Facilities: Open Air, Security Patrol, Lighting
View Details
Central Business District
University Campus Parking
Airport Economy Parking
localhost/smart-parking/venue.html
ENG
IN
01:01
06-11-2025

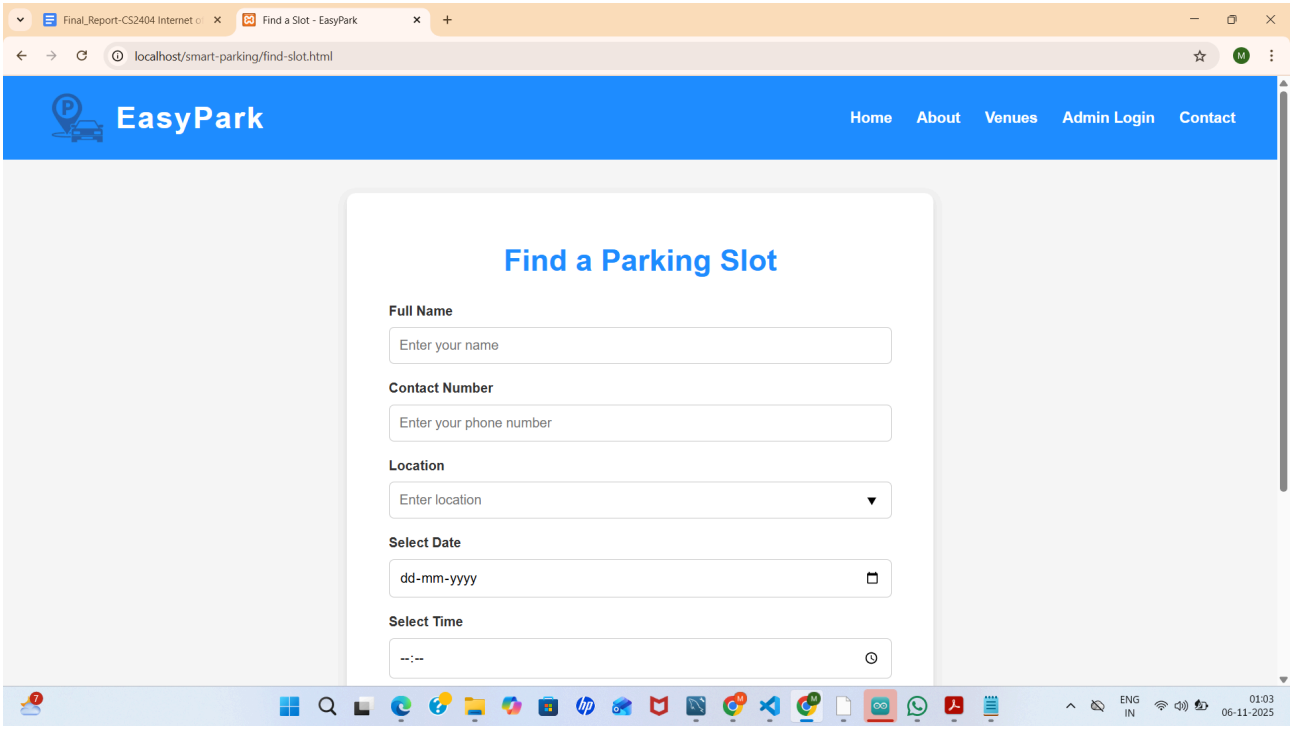

Final_Report-CS2404 Internet
Venues - EasyPark
localhost/smart-parking/venue.html
Location: MG Road, Bangalore
Facilities: EV Charging, CCTV, 24/7 Access
View Details
Location: Hyderabad International Airport
Facilities: Covered Parking, Surveillance, Security
View Details
Location: Electronic City, Bangalore
Facilities: EV Charging, Reserved Slots
View Details
Downtown Plaza Garage
Location: Main Street, New York
Facilities: Covered Parking, Security Cameras, Handicap Access
View Details
Greenfield Shopping Center
Location: Greenfield Avenue, Chicago
Facilities: EV Charging, 24/7 Access, CCTV
View Details
Seaside Mall Parking
Location: Ocean Drive, Miami
Facilities: Open Air, Security Patrol, Lighting
View Details
Central Business District Lot
Location: Downtown, San Francisco
Facilities: Reserved Slots, Surveillance, EV Charging
View Details
University Campus Parking
Location: College Road, Boston
Facilities: Student Parking, Shuttle Access, CCTV
View Details
Airport Economy Parking
Location: Airport Road, Dallas
Facilities: Long-Term Parking, Security, Shuttle Service
View Details
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com
ENG
IN
01:01
06-11-2025

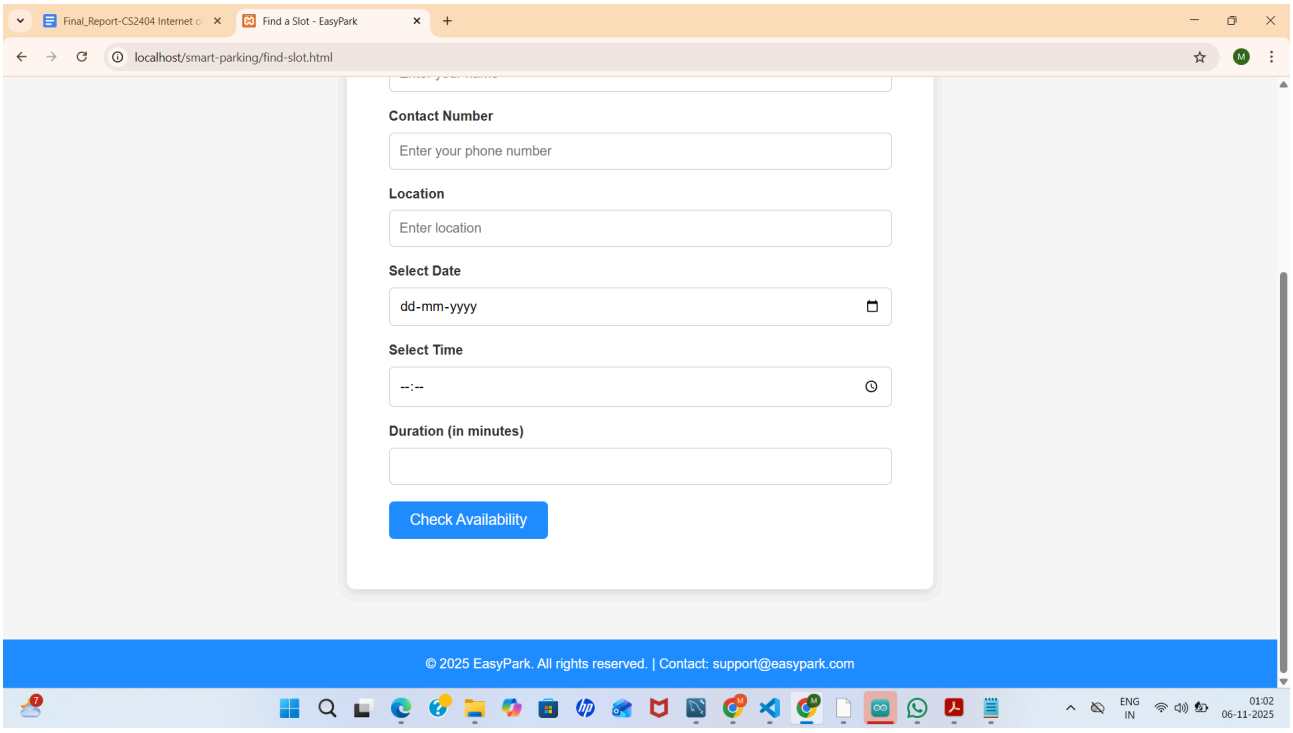

Final_Report-CS2404 Internet
EasyPark - Admin Login
localhost/smart-parking/adminlogin.html
EasyPark
Home
About
Venues
Admin Login
Contact
Venue Admin Login
Enter Venue Name
Enter Email
Enter Password
Login
New Venue? Register Here
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com
ENG
IN
01:01
06-11-2025

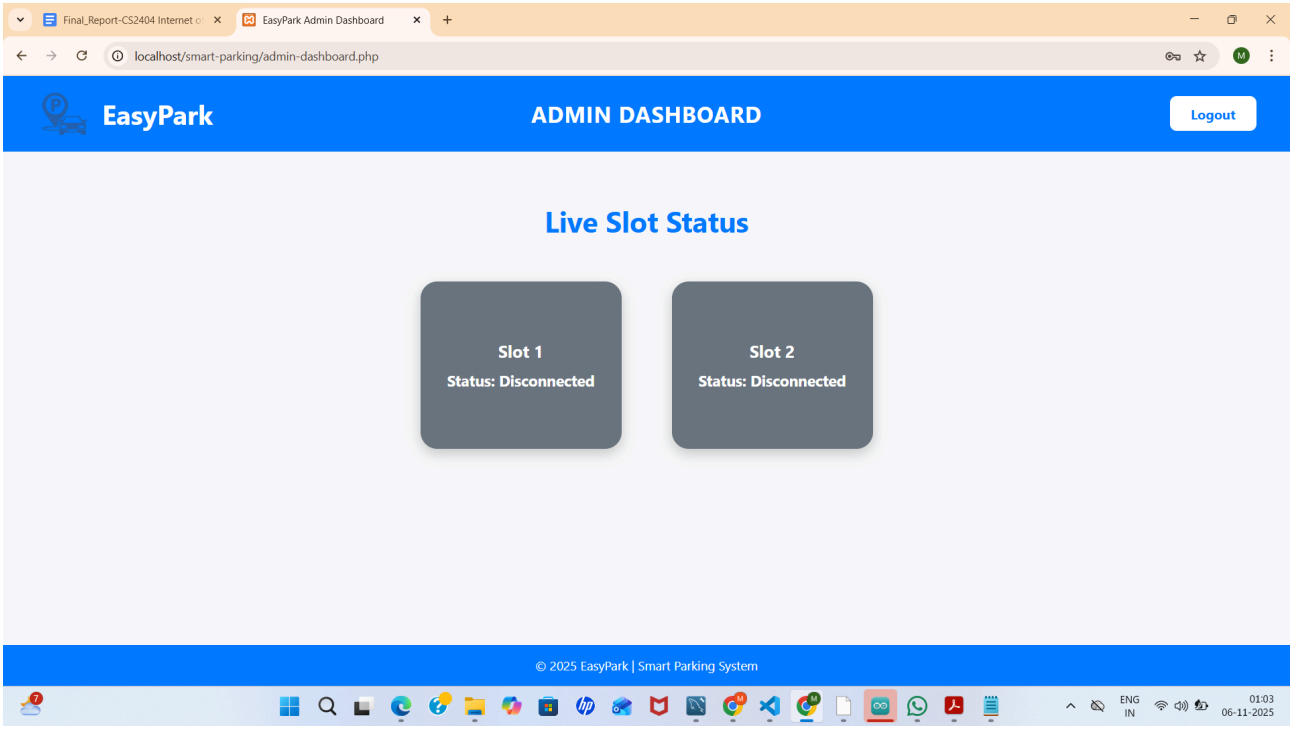

Final_Report-CS2404 Internet
Register Your Venue | EasyPark
localhost/smart-parking/register.html
Register Your Venue
Want to transform your parking area into a smart, IoT-powered space? EasyPark's Smart Parking System uses ultrasonic sensors, ESP32 modules, and LED indicators to monitor every parking slot in real-time.
Since the setup involves sensor placement, circuit configuration, and IoT integration, our dedicated technical team will handle the complete installation process for your venue. Simply contact us to get started — we'll guide you through every step!
Contact Our Setup Team
Email: setup@easypark.com
Phone: +91 98765 43210
Office: EasyPark Technologies Pvt. Ltd., Bengaluru, India
Go to Contact Page
ENG
IN
01:02
06-11-2025


Final_Report-CS2404 Internet
Contact Us | EasyPark Smart Par
localhost/smart-parking/contact.html
EasyPark
Home
About
Venues
Admin Login
Contact
Contact Us
Reach out to our EasyPark team for setup, technical support, or partnership inquiries.
Full Name
Email Address
Phone Number
Address of the Location
Your Message...
Send Message
ENG
IN
01:02
06-11-2025


Final_Report-CS2404 Internet
Contact Us | EasyPark Smart Par
localhost/smart-parking/contact.html
Email Address
Phone Number
Address of the Location
Your Message...
Send Message
Contact Information
Email: setup@easypark.com
Phone: +91 98765 43210
Address: EasyPark Technologies Pvt. Ltd., Bengaluru, India
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com
ENG
IN
01:02
06-11-2025


Final_Report-CS2404 Internet
Find a Slot - EasyPark
localhost/smart-parking/find-slot.html
EasyPark
Home
About
Venues
Admin Login
Contact
Find a Parking Slot
Full Name
Enter your name
Contact Number
Enter your phone number
Location
Enter location
Select Date
dd-mm-yyyy
Select Time
--:--
ENG
IN
01:03
06-11-2025


Final_Report-CS2404 Internet
Find a Slot - EasyPark
localhost/smart-parking/find-slot.html
Contact Number
Enter your phone number
Location
Enter location
Select Date
dd-mm-yyyy
Select Time
--:--
Duration (in minutes)
Check Availability
© 2025 EasyPark. All rights reserved. | Contact: support@easypark.com
ENG
IN
01:02
06-11-2025


Final_Report-CS2404 Internet
EasyPark Admin Dashboard
localhost/smart-parking/admin-dashboard.php
EasyPark
ADMIN DASHBOARD
Logout
Live Slot Status
Slot 1
Status: Disconnected
Slot 2
Status: Disconnected
© 2025 EasyPark | Smart Parking System
ENG
IN
01:03
06-11-2025

APPENDIX 2 : SOURCE CODE

MYSQL

```
CREATE DATABASE IF NOT EXISTS easypark;
USE easypark;

CREATE TABLE IF NOT EXISTS reservations (
   id INT AUTO_INCREMENT PRIMARY KEY,
   full_name VARCHAR(100) NOT NULL,
   contact VARCHAR(15) NOT NULL,
   location VARCHAR(50) NOT NULL,
   date DATE NOT NULL,
   time TIME NOT NULL,
   duration INT NOT NULL,
   assigned_slot VARCHAR(20),
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE venue_admins (
   id INT AUTO_INCREMENT PRIMARY KEY,
   venue_name VARCHAR(100) NOT NULL,
   email VARCHAR(100) NOT NULL UNIQUE,
   password VARCHAR(255) NOT NULL
);
INSERT INTO venue_admins (venue_name, email, password)
VALUES ('Nexon Mall', 'nexon@easypark.com', 'admin123');
```

INO FILE

```
#include <WiFi.h>
#include <WebServer.h>

// Wi-Fi credentials
const char* ssid = "Spark";
const char* password = "123456789";

WebServer server(80);
```

```
// Pins for both slots
const int trigPin1 = 12, echoPin1 = 13, redLed1 = 14, yellowLed1 = 27, greenLed1 = 26;
const int trigPin2 = 4, echoPin2 = 15, redLed2 = 21, yellowLed2 = 23, greenLed2 = 22;

// Reservation state
unsigned long slot1End = 0, slot2End = 0;
bool slot1Reserved = false, slot2Reserved = false;

// Distance function
float readDistanceCM(int trig, int echo) {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long duration = pulseIn(echo, HIGH, 30000);
  float distance = duration * 0.034 / 2;
  if (distance == 0 || distance > 400) distance = 400;
  return distance;
}

void setup() {
  Serial.begin(115200);

  pinMode(trigPin1, OUTPUT); pinMode(echoPin1, INPUT);
  pinMode(trigPin2, OUTPUT); pinMode(echoPin2, INPUT);

      pinMode(redLed1, OUTPUT); pinMode(yellowLed1, OUTPUT); pinMode(greenLed1,
OUTPUT);
      pinMode(redLed2, OUTPUT); pinMode(yellowLed2, OUTPUT); pinMode(greenLed2,
OUTPUT);

  digitalWrite(greenLed1, HIGH);
  digitalWrite(greenLed2, HIGH);
```

```
Serial.println("Connecting to Wi-Fi...");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
Serial.println("\nConnected to Wi-Fi!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

// Slot 1 reservation endpoint
server.on("/slot1/on", []() {
  int duration = server.arg("duration").toInt();
  if (duration <= 0) duration = 5; // default 5 mins

  slot1Reserved = true;
  slot1End = millis() + (unsigned long)duration * 60000UL;

  digitalWrite(yellowLed1, HIGH);
  digitalWrite(redLed1, LOW);
  digitalWrite(greenLed1, LOW);

  Serial.println("Slot 1 reserved for " + String(duration) + " minutes.");
  server.send(200, "text/plain", "Slot 1 reserved for " + String(duration) + " minutes.");
});

// Slot 2 reservation endpoint
server.on("/slot2/on", []() {
  int duration = server.arg("duration").toInt();
  if (duration <= 0) duration = 5;

  slot2Reserved = true;
  slot2End = millis() + (unsigned long)duration * 60000UL;

  digitalWrite(yellowLed2, HIGH);
  digitalWrite(redLed2, LOW);
```

```
    digitalWrite(greenLed2, LOW);

    Serial.println("Slot 2 reserved for " + String(duration) + " minutes.");
    server.send(200, "text/plain", "Slot 2 reserved for " + String(duration) + " minutes.");
  });

server.on("/status", []() {
  // Read LED states directly
  bool red1 = digitalRead(redLed1);
  bool yellow1 = digitalRead(yellowLed1);
  bool green1 = digitalRead(greenLed1);

  bool red2 = digitalRead(redLed2);
  bool yellow2 = digitalRead(yellowLed2);
  bool green2 = digitalRead(greenLed2);

  String slot1Status, slot2Status;

  // Determine slot1 status
  if (red1) slot1Status = "occupied";
  else if (yellow1) slot1Status = "reserved";
  else if (green1) slot1Status = "empty";
  else slot1Status = "disconnected";

  // Determine slot2 status
  if (red2) slot2Status = "occupied";
  else if (yellow2) slot2Status = "reserved";
  else if (green2) slot2Status = "empty";
  else slot2Status = "disconnected";

  String json = "{";
  json += "\"slot1\":\"" + slot1Status + "\",";
  json += "\"slot2\":\"" + slot2Status + "\"";
  json += "}";

  server.send(200, "application/json", json);
```

```
});


  server.begin();
  Serial.println("Web Server started!");
}

void loop() {
  server.handleClient();
  unsigned long now = millis();

  // Read distances
  float d1 = readDistanceCM(trigPin1, echoPin1);
  float d2 = readDistanceCM(trigPin2, echoPin2);

  bool car1 = d1 < 5;
  bool car2 = d2 < 5;

  // SLOT 1 logic
  if (slot1Reserved) {
    if (now >= slot1End) {
      slot1Reserved = false;
      digitalWrite(yellowLed1, LOW);
      digitalWrite(redLed1, LOW);
      digitalWrite(greenLed1, HIGH);
    } else {
      if (car1) {
        digitalWrite(redLed1, HIGH);
        digitalWrite(yellowLed1, LOW);
      } else {
        digitalWrite(redLed1, LOW);
        digitalWrite(yellowLed1, HIGH);
      }
      digitalWrite(greenLed1, LOW);
    }
  } else {
```

```
  if (car1) {
    digitalWrite(redLed1, HIGH);
    digitalWrite(greenLed1, LOW);
    digitalWrite(yellowLed1, LOW);
  } else {
    digitalWrite(greenLed1, HIGH);
    digitalWrite(redLed1, LOW);
    digitalWrite(yellowLed1, LOW);
  }
}

// SLOT 2 logic
if (slot2Reserved) {
  if (now >= slot2End) {
    slot2Reserved = false;
    digitalWrite(yellowLed2, LOW);
    digitalWrite(redLed2, LOW);
    digitalWrite(greenLed2, HIGH);
  } else {
    if (car2) {
      digitalWrite(redLed2, HIGH);
      digitalWrite(yellowLed2, LOW);
    } else {
      digitalWrite(redLed2, LOW);
      digitalWrite(yellowLed2, HIGH);
    }
    digitalWrite(greenLed2, LOW);
  }
} else {
  if (car2) {
    digitalWrite(redLed2, HIGH);
    digitalWrite(greenLed2, LOW);
    digitalWrite(yellowLed2, LOW);
  } else {
    digitalWrite(greenLed2, HIGH);
    digitalWrite(redLed2, LOW);
```

```
    digitalWrite(yellowLed2, LOW);
   }
  }

 delay(300);
}
```

Admin Dashboard (admin-dashboard.php)

```
<?php
session_start();
if (!isset($_SESSION['venueName'])) {
   header("Location: adminlogin.html");
   exit();
}

$venueName = $_SESSION['venueName'];
?>

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title><?= $venueName ?> - Smart Parking Dashboard</title>
<style>
 body { font-family: Arial, sans-serif; background-color: #f5f5f5; color: #333; }
   header { background-color: #007bff; color: white; padding: 15px 40px; text-align: center; font-size: 24px; }
  .slot { width: 150px; height: 150px; margin: 20px; display: inline-block; border-radius: 15px; text-align: center; vertical-align: top; }
 .slot.empty { background-color: #28a745; color: white; }
 .slot.reserved { background-color: #ffc107; color: black; }
 .slot.occupied { background-color: #dc3545; color: white; }
  .logout { position: absolute; top: 20px; right: 40px; background-color: white; border: none; color: #007bff; font-weight: bold; padding: 10px 25px; border-radius: 8px; cursor: pointer; }
</style>
```

```
</head>
<body>

<header><?= $venueName ?> - Smart Parking Dashboard</header>
<button class="logout" onclick="location.href='logout.php'">Logout</button>

<div class="dashboard">
  <div id="slot1" class="slot empty">
    <h3>Slot 1</h3>
    <p>Status: Empty</p>
  </div>
  <div id="slot2" class="slot empty">
    <h3>Slot 2</h3>
    <p>Status: Empty</p>
  </div>
  <!-- More slots as needed -->
</div>

<script>
function fetchSlotStatus() {
  fetch('esp_proxy.php', { cache: 'no-store' })
    .then(response => response.json())
    .then(data => {
      ['slot1', 'slot2'].forEach(slotId => {
        const slot = document.getElementById(slotId);
        const status = data[slotId];
        slot.className = 'slot ' + status;
        slot.querySelector('p').innerText = 'Status: ' + status.charAt(0).toUpperCase() + status.slice(1);
      });
    });
}
setInterval(fetchSlotStatus, 3000);
fetchSlotStatus();
</script>

</body>
```

```
</html>
```

Reservation Backend (reserve.php)

```
<?php
$servername = "localhost";
$username = "root";
$password = "mouman321";
$dbname = "easypark";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}

$name = $_POST['name'];
$contact = $_POST['contact'];
$location = $_POST['location'];
$date = $_POST['date'];
$time = $_POST['time'];
$duration = intval($_POST['duration']);

$statusjson = file_get_contents("http://esp32-ip/status"); // replace with actual ESP32 IP
$liveStatus = json_decode($statusjson, true);

$availableSlots = ["Slot 1", "Slot 2"];
foreach ($availableSlots as $slot) {
   if ($liveStatus[strtolower(str_replace(' ', '', $slot))] == 'occupied') {
      continue;
   }
   // Check overlapping reservations
    $sql = "SELECT * FROM reservations WHERE location='$location' AND date='$date' AND
assigned_slot='$slot' AND (
        (start_time <= '$time' AND end_time > '$time') OR
           (start_time < DATE_ADD('$time', INTERVAL $duration MINUTE) AND end_time >=
DATE_ADD('$time', INTERVAL $duration MINUTE))
```

```
        )";
    $result = $conn->query($sql);
    if ($result->num_rows == 0) {
        $startTime = $date . ' ' . $time;
        $endTime = date('Y-m-d H:i:s', strtotime("+$duration minutes", strtotime($startTime)));
         $insertSql = "INSERT INTO reservations (name, contact, location, assigned_slot, start_time, end_time)
                    VALUES ('$name', '$contact', '$location', '$slot', '$startTime', '$endTime')";
        $conn->query($insertSql);
        echo "Reservation successful for $slot!";
        break;
    }
}
$conn->close();
?>
```

ESP32 Proxy (esp_proxy.php)

```
<?php
header('Content-Type: application/json');
$esp32url = 'http://10.155.156.210/status'; // ESP32 IP address
$response = file_get_contents($esp32url);
if ($response === FALSE) {
    echo json_encode(['slot1'=>'disconnected', 'slot2'=>'disconnected']);
} else {
    echo $response;
}
?>
```

User Reservation Form (find-slot.html)

```
<form action="reserve.php" method="POST" style="max-width: 700px; margin: auto; text-align: left;">
     <label for="name" style="font-weight: bold; display: block; margin-bottom: 8px;">Full Name</label>
    <input type="text" id="name" name="name" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
<label for="contact" style="font-weight: bold; display: block; margin-bottom: 8px;">Contact Number</label>
```

```
  <input type="tel" id="contact" name="contact" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
<label for="location" style="font-weight: bold; display: block; margin-bottom: 8px;">Location</label>
    <select id="location" name="location" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
  <option value="">Select Venue</option>
  <option value="Nexon Mall">Nexon Mall</option>
  <option value="City Center Plaza">City Center Plaza</option>
 </select>
<label for="date" style="font-weight: bold; display: block; margin-bottom: 8px;">Select Date</label>
   <input type="date" id="date" name="date" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
    <label for="time" style="font-weight: bold; display: block; margin-bottom: 8px;">Select Time</label>
   <input type="time" id="time" name="time" required style="width: 100%; padding: 10px; margin-bottom: 15px;">
  <label for="duration" style="font-weight: bold; display: block; margin-bottom: 8px;">Duration (minutes)</label>
 <input type="number" id="duration" name="duration" min="1" max="60" required style="width: 100%; padding: 10px; margin-bottom: 20px;">
  <button type="submit" style="background-color: #1e90ff; color: white; padding: 12px 30px; border: none; border-radius: 6px; cursor: pointer; font-size: 16px;">Reserve Slot</button>
</form>
```

This consolidated code represents the core backend and frontend logic of your EasyPark system and can be directly included in your project report source code appendix or implementation sections.

APPENDIX 3 : PLAGIARISM DETAILS

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS

Plagiarism	0%	Partial Match	0%
Exact Match	0%	Unique	100%